Smart Agricultural Technology 13 (2026) 101872

Contents lists available at ScienceDirect

Smart Agricultural Technology

journal homepage: www.journals.elsevier.com/smart-agricultural-technology

An agent-Based service architecture for smart greenhouses: Telemetry analytics and decision support with RAG-grounded LLM agents

Sofía Pardo-Pina [a], Jörn Germer [b], Ricardo Suay-Cortés [c], Manuel Platero-Horcajadas [d], Francisco-Javier Ferrández-Pastor [d,*]

[a] Miguel Hernandez University, Institute for Agro-food and Agro-environmental Research and Innovation, Carretera de Beniel, 03312, Spain
[b] University of Hohenheim, Department of Crop Water Stress Management in the Tropics and Subtropics, Stuttgart, 70599, Germany
[c] Institute of Agrochemistry and Food Technology, Spanish National Research Council (CSIC), Paterna, 46980, Spain
[d] University of Alicante,Department of Computer Science and Technology, San Vicente del Raspeig, 03690, Spain

ARTICLE INFO

Keywords:
Hydroponics
IoT
Intelligent agents
Greenhouse automation
Low-Code orchestration

ABSTRACT

This work addresses a practical barrier in smart greenhouse operations: although IoT platforms provide telemetry and dashboards, growers often lack direct, operational access to AI capabilities that can convert sensor streams into actionable decisions and *bounded* automated interventions. We propose an AI-driven, layer-based IoT architecture integrated with an Agent-Based Service Architecture (ASA) that orchestrates specialized agents to move from telemetry to decisions. The ASA comprises: (S1) a conversational Chat-Agent grounded via Retrieval-Augmented Generation (RAG) over standard operating procedures and local context; (S2-IoT) a telemetry analysis service that computes agronomic indicators and triggers bounded actuation; and (S2-E) an incident/emergency assessment service that recommends responses *and can trigger bounded actions when enabled by the operator*. Crucially, automated actions are executed only when they satisfy predefined operating conditions and remain consistent with operator-defined safety policies (stage-dependent thresholds, validation gates, and conflict-resolution rules). The services are orchestrated through auditable low-code workflows (n8n in the cloud and raspberrypi/Home Assistant at the edge) and include an edge-first offline fallback when cloud orchestration is unavailable. The system is deployed and validated in a 50 m^2 controlled-environment hydroponic greenhouse cultivating industrial hemp. For conversational support, GPT-5.2 answers 38/40 domain questions correctly (95.0%), outperforming GPT-4o-Mini (32/40; 80.0%), with ROUGE-L = 0.272, BLEU-4 = 3.821, and BERTScore = 0.872. S2-IoT estimates daily light integral (DLI) with MAE = 0.005 and RMSE = 0.008 mol m^{-2} d^{-1} over 13 evaluated days. S2-E attains 0.81 accuracy and 0.97 stability over 36 emergency test cases `status` (OK/WARNING/ALARM) . Overall, results indicate that agent-orchestrated LLM-RAG services can provide growers with direct AI-enabled operational functions-monitoring, decision support, and policy-compliant actuation-within IoT greenhouse workflows. Further multi-site and multi-season evaluation is planned as future work.

1. Introduction

By integrating Internet of Things (IoT)-based sensing and actuation, growers can monitor and adjust key variables such as photosynthetically active radiation (PAR), daily light integral (DLI), vapor pressure deficit (VPD), and evapotranspiration (ET) in real time [1,2].

This work proposes a **generalized, intelligent, agent-orchestrated IoT architecture** for greenhouse cultivation that can be adapted to diverse production contexts. Although the architecture is presented using a hydroponic installation as the primary *case study*, the model is inherently modular and transferable to other cultivation methods, with only the lower layers—sensing, actuation, and immediate control—requiring adaptation to the specific production technique.

When defining such a model, it is necessary to start from a concrete infrastructure to determine operational constraints, data flows, and service interactions. Hydroponics was selected as the demonstration platform because of its stringent requirements for precise nutrient and water management, making it an ideal scenario for evaluating orchestration strategies and intelligent agent services.

For other cultivation modalities, adaptation primarily affects the Physical Layer: soil-based systems replace nutrient-solution pH/EC and NFT-specific telemetry with soil moisture (VWC), soil temperature, and

* Corresponding author.
E-mail address: fjferran@ua.es (F.-J. Ferrández-Pastor).

https://doi.org/10.1016/j.atech.2026.101872
Received 7 December 2025; Received in revised form 8 February 2026; Accepted 11 February 2026
Available online 20 February 2026

optionally nutrient/EC probes, while aeroponic systems require pressure, nozzle-health, and high-frequency pulse timing signals. Local logic is correspondingly updated (e.g., irrigation triggers based on VWC hysteresis, or nozzle fault detection), whereas the orchestration and service layers remain unchanged.

The proposed system is **user-centered by design**, involving growers in the definition of requirements, workflows, and service priorities. Moreover, it leverages **open-source, low-cost technologies** whenever feasible, aiming to reduce adoption barriers and ensure scalability for small- and medium-scale producers.

The architecture includes two main service types: (i) a conversational agent (Chat-Agent) that assists growers in planning, supervision, and decision-making, and (ii) an IoT monitoring and control agent (IoT-Agent) capable of detecting anomalies and, under well-defined operational and safety constraints, initiating autonomous corrective actions. This dual-service approach is intended to enhance responsiveness, improve resource efficiency, and provide a scalable framework for AI-assisted greenhouse management.

Problem. Agronomic installations generate heterogeneous, high-frequency telemetry encompassing environmental, irrigation/fertigation, and energy parameters. Converting these data streams into timely actions—such as alerts, safe automated actuation, or user-friendly operational guidance—remains challenging without a modular intelligence layer capable of integrating sensing, decision-making, and control.

Approach. In response to these challenges, we propose an Agent-Based Service Architecture (ASA) layered over the greenhouse IoT stack. The ASA service layer is designed to host multiple types of intelligent services aimed at enabling user interaction and optimizing responses to alerts, events, and agronomic processes. In this paper, we implement and evaluate two exemplar services: (S1) a farmer-facing conversational agent and (S2) a telemetry-driven monitoring and actuation agent. A proof-of-concept (PoC) is carried out on an industrial hemp hydroponic crop.

Contributions. The main contributions of this work are as follows:

- **Generalized architecture model:** We present a modular Agent-Based Service Architecture (ASA) layered over a greenhouse IoT stack, adaptable to different agronomic installations and cultivation techniques.
- **Integration of heterogeneous services:** The ASA service layer is designed to host multiple intelligent services, enabling both user interaction and autonomous optimization of agronomic processes.
- **Dual-service proof of concept:** Two representative services are implemented and evaluated: a farmer-facing conversational agent (S1) and a telemetry-driven monitoring and actuation agent (S2).
- **User-centered, open-source approach:** The system design incorporates grower feedback and prioritizes open-source, low-cost technologies to improve accessibility for small- and medium-scale producers.
- **Validation in a real cultivation scenario:** The proposed architecture is experimentally validated on an industrial hemp hydroponic crop, demonstrating responsiveness, adaptability, and potential for broader adoption in controlled-environment agriculture.

To make the novelty explicit, we now distinguish: (i) the *Agent-Based Service Architecture (ASA)* as a service-layer pattern that separates advisory (S1) and actuation (S2/S2-E) responsibilities; (ii) an *AI-driven orchestration* approach that operationalizes these services via reproducible low-code workflows (n8n) and auditable policy gates; (iii) a *dual-service implementation* that couples a farmer-facing RAG Chat-Agent with a telemetry-driven monitoring/actuation agent, enabling end-to-end assistance beyond dashboards; and (iv) a safety- and resiliency-oriented execution model (sensor validation + threshold hysteresis + offline fallback) that keeps edge controllers functional during connectivity loss. We additionally strengthen positioning relative to recent greenhouse AI/IoT literature in the Related Work and Discussion sections [3–5].

Paper organization.

The remainder of this paper follows a conventional scientific structure. Section 1 introduces the problem, scope, related work, and contributions. Section 2 describes the materials and methods, including the proposed architecture, AI components, safety mechanisms, and the hydroponic greenhouse experimental setup. Section 3 reports the experimental results and validation of the platform and intelligent agents. Section 4 discusses implications, limitations, and deployment trade-offs. Section 5 concludes the paper and summarizes future research directions.

1.1. Related work

Precision agriculture and the role of AI and IoT

Precision Agriculture (PA) is a management strategy that utilizes observation, measurement, and targeted action to address inter- and intra-crop variability within agricultural plots [6]. It involves collecting, processing, and analyzing temporal, spatial, and individual data, and combining these with other sources of information to support management decisions [7]. The primary goals of PA are to optimize resource-use efficiency, enhance productivity and quality, increase profitability, and promote the sustainability of agricultural production [7,8]. PA is recognized as a cornerstone of sustainable agriculture, aiming to ensure perennial food production while respecting ecological, economic, and social boundaries [6].

Artificial Intelligence (AI) and the Internet of Things (IoT) are fundamental to recent advances in precision agriculture, driving significant transformations across the sector [6,8–10].

- **AI's contribution**: In controlled-environment agriculture and hydroponics, AI is increasingly used to support closed-loop climate and fertigation management, including (i) data-driven detection of sensor faults and abnormal environmental states [11,12], (ii) prediction and optimization of energy- and resource-efficient setpoints (temperature, humidity, CO_2, light, irrigation) [12,13], and (iii) decision support grounded in curated operational documents (e.g., SOPs) and deployment constraints to improve reliability and compliance in practice [14,15]. Recent work also highlights a shift from isolated prediction models to system-level architectures that combine real-time telemetry, knowledge bases, and operational workflows to improve robustness and usability in greenhouse deployments [12,16,17].
- **IoT's contribution**: IoT devices are essential for collecting real-time data on critical variables such as temperature, humidity, soil moisture, and pH levels [6,8]. This information supports informed decision-making and enables automated operations [8]. IoT-based monitoring systems are widely used in greenhouse management [2] and enable continuous monitoring and anomaly detection in smart farms [18]. Sensors, as key IoT components, also measure soil resistivity and local climatic conditions [6].

Smart greenhouses and hydroponic cultivation

Greenhouses in 2025 are increasingly described as intelligent ecosystems, controlled by smart systems and powered by real-time data, IoT devices, cloud platforms, and AI [19]. This evolution signifies a clear shift from manual operations towards highly data-driven environments [19]. AI-powered environmental control systems contribute to resilient and efficient greenhouse farming by optimizing climatic conditions and resource usage [20].

In hydroponic systems, IoT-based smart greenhouse platforms enable automated monitoring and control of parameters such as water pH, nutrient concentration, light, and temperature, often via mobile and web applications [21]. These systems offer alternative solutions to food shortages exacerbated by climate change and land scarcity [21]. Sensor networks are widely recognized as crucial for controlled-environment agriculture, including hydroponics [1].

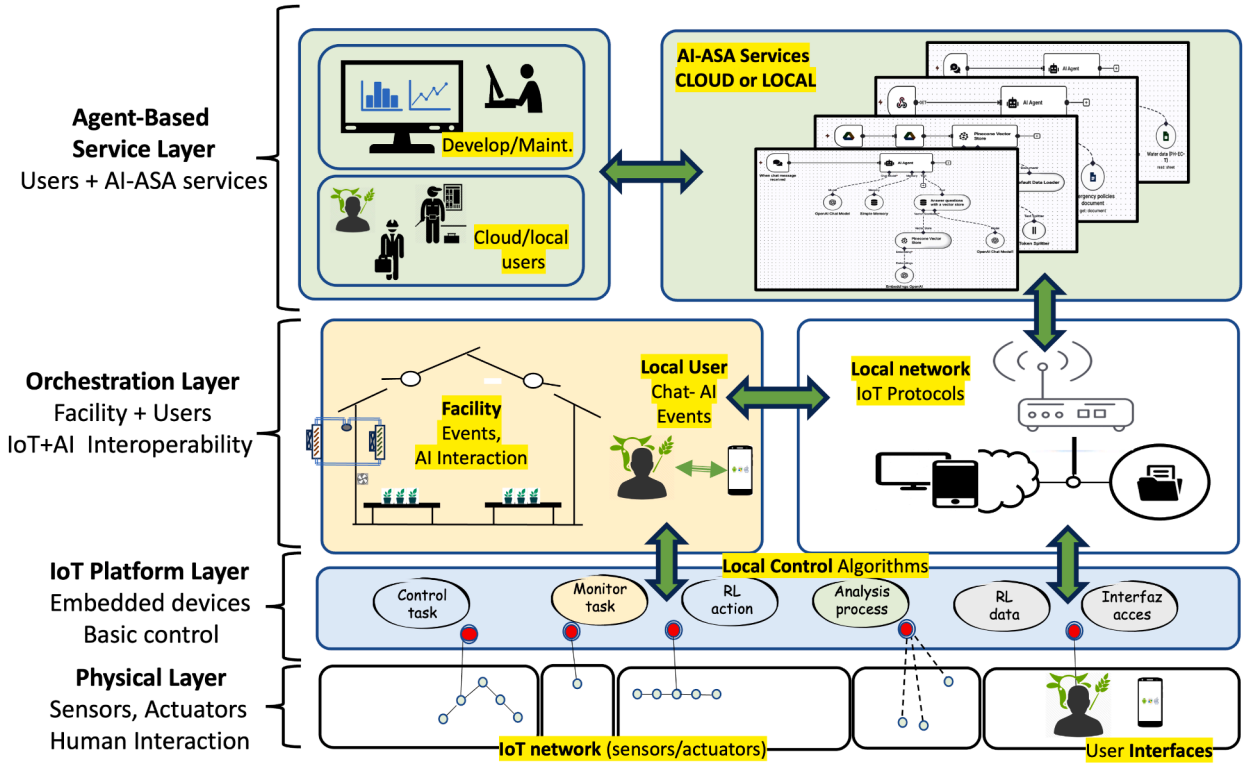


Fig. 1. Layered model of the agent-orchestrated hydroponic system. From bottom to top: IoT network, local control algorithms, facility/AI control, local network and protocols, AI services (cloud or local), and user interfaces (local and cloud).

Key equipment commonly found in smart greenhouses includes:

- **Sensors**: Used for measuring soil or substrate moisture, ambient temperature, CO_2 levels, light spectrum (PAR), and humidity [19].
- **Controllers and actuators**: Programmable Logic Controllers (PLCs), Arduino/Raspberry Pi boards, and Modbus- or MQTT-based logic units that manage motorized vents, irrigation valves, misting systems, fans, and lighting [19].
- **Gateways**: Devices such as LoRaWAN hubs, Zigbee-Wi-Fi bridges, and MQTT brokers that connect sensors and actuators to local or cloud platforms [19].
- **Cameras**: Employed for crop-health imaging, visual inspection, and AI-driven growth tracking [19].
- **Dashboards**: Web and mobile user interfaces used for monitoring real-time data, visualizing historical trends, and receiving alerts [19].

Real-world applications of automation in greenhouses have reported significant benefits, including increased yields (up to 20 times per square meter), substantial reductions in water usage (up to 90% less), and decreased labor costs (up to 30%) [19].

AI-Driven agent-orchestrated IoT architecture: a conceptual framework

Although the exact term "agent-orchestrated" is not uniformly used across recent agricultural IoT literature, the underlying principles of autonomous, coordinated, and centrally managed AI-driven systems are closely aligned with the proposed architecture. These principles are explored and implicitly supported in several related contexts.

In a broader sense, AI is envisioned as the orchestrator of integrated, personalized, and predictive ecosystems [22]. This vision involves AI evolving into a central agent that shifts systems from reactive to proactive and preventive models [22]. The "Restructuring Stage" of AI transformation, originally described in healthcare, suggests a fundamental redesign of systems in which AI capabilities become core, and AI assumes primary responsibility for tasks traditionally performed by humans [22]. This conceptualization is highly applicable to advanced greenhouse cultivation, where AI systems could autonomously manage and optimize complex networks of IoT devices and cultivation processes.

The growing interest in low-code orchestration for industrial IoT [23] further supports the feasibility of creating flexible and adaptable frameworks for managing sophisticated IoT environments. This aligns with the notion of an "agent-orchestrated" approach by enabling streamlined integration and management of heterogeneous devices and services. Moreover, the development of decentralized AI platforms that connect and coordinate networks of urban farms underscores a trend towards distributed orchestration for knowledge sharing and resource optimization [9]. Reviews of AI-enabled IoT systems for controlled-environment agriculture also highlight the integrated and interconnected nature of these technologies [10]. In parallel, chatbots and conversational interfaces are emerging in agriculture as natural entry points to such orchestrated systems [24].

Challenges and the Need for a Human-Centered Approach

The widespread implementation of advanced AI-driven systems in agriculture faces several challenges. One concern is the potential concentration of power among large corporations, driven by the high cost and complexity of sophisticated AI technologies. This dynamic could deepen the digital divide, disadvantaging smaller agricultural producers and homogenizing the food landscape [9].

Furthermore, an excessive reliance on complex and opaque algorithms can lead to algorithmic dependence, making cultivation systems vulnerable to cyberattacks, software glitches, and inherent algorithmic biases [9]. A single point of failure in a centralized AI infrastructure could have cascading effects, potentially leading to widespread crop failures [9].

To mitigate these risks and ensure equitable and sustainable development, a human-centered approach to technological innovation is

Table 1
Feature-level comparison with representative prior directions (qualitative, because experimental setups differ).

Capability	IoT monitoring	Data Layer control models	This work (ASA)
Low-code orchestration workflow	–	–	✓
Human Conversational guidance	–	–	✓
Safety-gated actuation agent	limited	model-specific	✓
Offline fallback (edge-first)	variable	variable	✓
Quantitative validation in industrial setting	variable	variable	✓

crucial [9]. This perspective advocates for the democratization of technology, promoting open-source AI platforms and the development of affordable, user-friendly tools that enable broad participation across all scales of agricultural production [9]. The ultimate goal is to foster a symbiosis in which AI functions as a powerful partner to human intelligence, providing real-time data and insights while allowing farmers to focus on the more creative and strategic aspects of their work. This balanced approach seeks to ensure that technology genuinely supports communities and contributes to a more just and sustainable food system [9].

Advances in smart farming have demonstrated the potential of combining hydroponic systems with AI-driven monitoring to improve yields and sustainability [25]. Conversational interfaces allow growers to interact naturally with control systems [24], while anomaly-detection agents can automate responses to environmental deviations [18]. Low-code orchestration platforms, such as Node-RED, Flowise, and n8n, facilitate flexible and scalable workflows for integrating multiple data sources and services [23]. However, comprehensive implementations in hydroponic cannabis or industrial hemp production remain limited.

Current literature underscores the value of AI-enabled IoT in hydroponics for environmental optimization and operational efficiency [10]. Yet, end-to-end deployments that combine conversational guidance with closed-loop actuation in hydroponic settings are rare. Our proof of concept (PoC) suggests that a layered agent-based approach can reduce response times and improve explainability while keeping integration effort low.

1.2. Positioning relative to prior greenhouse AI/IoT systems

Recent surveys summarize advances in smart-greenhouse monitoring, prediction, and automation [3,4,26,27]. Control-oriented works increasingly leverage deep learning (e.g., Transformer-based models) to learn environment–control relationships and enable more direct control policies [5,28,29]. In parallel, crop-specific prediction models (e.g., irrigation prediction for greenhouse tomatoes) demonstrate the value of combining sensor streams with learned models for actionable decisions [30].

In contrast to many prior systems that focus on either (i) sensor monitoring and dashboards or (ii) a single predictive/control model, our contribution is an integrated *service architecture* that combines a retrieval-grounded conversational service (S1) with safety-gated monitoring/actuation services (S2/S2-E), orchestrated through an auditable workflow layer. Because evaluation protocols, crops, and sensors differ across studies, we add a feature-level comparison and explicitly discuss where direct quantitative comparisons are not meaningful. Table 1 summarizes the capabilities that distinguish our ASA-based approach from representative IoT monitoring systems and learning-based greenhouse control models.

2. Materials and methods

The proposed architecture is a generalizable model for intelligent greenhouse control that separates concerns into layers and defines explicit data flows, interaction patterns, and services (Fig. 1). It supports both local and cloud execution and enables safe, auditable actuation under well-defined operational and safety constraints.

2.1. Physical layer (sensors and actuators)

This layer comprises a cost-effective infrastructure of environmental and process sensors (e.g., air temperature, relative humidity, PAR/PPFD, lux, solar radiation, CO_2, water temperature, pH, electrical conductivity (EC), tank/load-cell mass, flow, and pressure) and actuators (e.g., pumps, valves, nutrient dosing systems, fans, vents, photovoltaic shade cloths, and supplemental lighting). To enhance reliability when using low-cost hardware, multiple sensors can be deployed for the same variable, enabling cross-checking of readings and early detection of drifts or failures.

All sensors and actuators expose communication interfaces based on standardized IoT protocols (e.g., MQTT, CoAP, or HTTP/REST), ensuring that their data and control functions can be integrated seamlessly into the broader architecture. Likewise, all subsystems of the installation—such as irrigation, lighting, ventilation, and climate control—are equipped with communication interfaces that make them interoperable within the system, enabling unified management and coordinated decision-making across services.

The specific configuration of the sensor and actuator network is determined by the cultivation method—whether hydroponic, pot-based, or alternative greenhouse production systems—ensuring that the hardware layout matches the operational requirements of each installation. Devices typically publish measurements at intervals ranging from 1 to 300 s and accept idempotent control commands with acknowledgements. Minimal in-device validation (e.g., range checks and rate-of-change limits) prevents corrupt or anomalous payloads from entering the system.

The IoT network can be supervised through basic user interfaces, allowing operators to configure devices, access real-time and historical data, and issue control commands under human oversight, thus ensuring both safety and operational compliance.

2.2. IoT platform layer and basic control algorithms

This layer hosts embedded control devices that acquire sensor data from the physical layer and execute local decision-making logic. Typical devices include ESP-based boards, Arduinos, and similar microcontrollers interconnected via wired or wireless links, forming a local device network with reactive programming capabilities. This network supports direct peer-to-peer communication, coordinated actions, and full interoperability across all subsystems of the installation (e.g., irrigation, lighting, ventilation, climate control).

Each embedded device provides a user interaction interface for monitoring, control, configuration, and management of deployed hardware. These interfaces allow operators to perform all required actions directly, ensuring a user-oriented approach that matches the needs and workflows of the targeted users.

A lightweight runtime operates close to the greenhouse edge, enabling low-latency processing and actuation. Typical functionalities include:

- **Basic monitoring tasks:** data validation, noise filtering, and gap filling; derivation of secondary variables such as PAR, DLI, VPD, tank slope (g/min), and rule-based features.

- **Basic control tasks:** execution of algorithms such as threshold-based control or PID loops (e.g., irrigation pulses, ventilation steps), while respecting hard operational bounds and cooldown periods.
- **Analysis processes:** event correlation and anomaly scoring; generation of compact data summaries for higher layers in the architecture.
- **Data storage for learning services:** collection of datasets that feed reinforcement learning policies or pattern-recognition models, enabling continuous improvement of control strategies and anomaly detection.

2.3. Orchestration layer: Local network and IoT protocols

This layer deploys a local communication network that ensures full interoperability among all subsystems of the installation and serves as the interface to the service-development layer, where AI-based services are implemented. The local network integrates all communication systems using standardized IoT protocols (e.g., MQTT, CoAP, OPC UA, HTTP/WebSocket), enabling seamless data exchange and coordinated actuation across heterogeneous devices and control domains.

To manage the installation locally, this layer integrates programmable devices running open-source and low-code platforms—such as Linux-based systems on PCs or Raspberry Pi boards, or specialized operating systems like Home Assistant. These devices orchestrate all subsystems (e.g., irrigation, lighting, climate control) and provide the bridge toward the upper service layer for AI-driven functionalities.

Ultimately, this layer manages the entire installation at the local level and executes the actions deployed by the AI-agent layer. As in all layers, intuitive interfaces should be developed, and users should be actively involved in the design and development of services to ensure usability and adoption.

Beyond providing connectivity, this layer executes advanced control and monitoring functions that operate across subsystems, aggregating data, orchestrating commands, and enforcing consistent operational policies throughout the installation.

2.3.1. Sensor validation, thresholding, and conflict resolution

To support safe and reliable actuation, the orchestration layer applies a two-stage validation pipeline before any agent recommendation can be executed. **(V1) Per-sensor validation:** each incoming measurement is checked for schema compliance (units and timestamp monotonicity), hard physical bounds, and rate-of-change limits; samples failing these checks are discarded and the sensor health flag is degraded. **(V2) Cross-sensor conflict resolution:** when multiple sensors observe the same variable (e.g., two or three T/RH probes), we compute a robust aggregate using the median m and a dispersion statistic (median absolute deviation, MAD). A reading x_i is considered consistent if $|x_i - m| \leq \tau \cdot \mathrm{MAD}$ (or within a fixed tolerance when MAD≈ 0). Outliers are quarantined for a cooldown period and excluded from aggregation, while the final value is set to the median of the remaining consistent sensors. If fewer than two consistent sensors remain, the system degrades to a last-known-good value, emits a `WARNING` status, and blocks high-impact actuation until redundancy is restored. Stage-dependent thresholding is then applied to the aggregated values using *hysteresis* and *persistence* guards: alarms require k consecutive violations (debouncing), and recovery requires sustained in-range readings, preventing chattering and reducing false positives. This logic is implemented in the local policy engine (Home Assistant automations) and mirrored in the S2/S2-E policy gate, ensuring that agent decisions always operate on validated data and consistent thresholds.

The main functions of this layer include:

- **Protocol integration:** unifying device and subsystem communications under interoperable IoT standards.
- **Data routing and preprocessing:** collecting telemetry from the IoT layer, performing validation, transformation, and routing toward service endpoints.
- **Distributed control:** coordinating multi-device actions across subsystems to respect operational and safety limits (setpoint ranges, rate limits, multi-sensor confirmation 2-of-3, conditioning times, rollback), with human-in-the-loop escalation for high-risk actions.
- **Real-time event handling:** detecting cross-system events and executing predefined or adaptive responses.
- **Edge processing and scheduled delivery:** persisting real-time telemetry at the edge and performing filtering, aggregation (e.g., rollups, downsampling), compression, and feature extraction; generating user-configurable periodic digests and reports for delivery at scheduled intervals, while prioritizing and forwarding critical or emergency events upstream for immediate analysis—independent of batch cadences—and applying local safeguards and actions within this layer.
- **Service interfacing:** exposing structured APIs and data streams to the upper layers for AI-driven analysis, decision-making, and optimization.
- **Security and access control:** managing authentication, authorization, and encrypted communications within the local network.

In this work, the farmer-facing Chat-Agent (S1) is implemented using a Retrieval-Augmented Generation (RAG) approach, in which a Large Language Model (LLM) is grounded with a vectorized knowledge base of domain documents and standard operating procedures (SOPs) to produce context-aware responses.

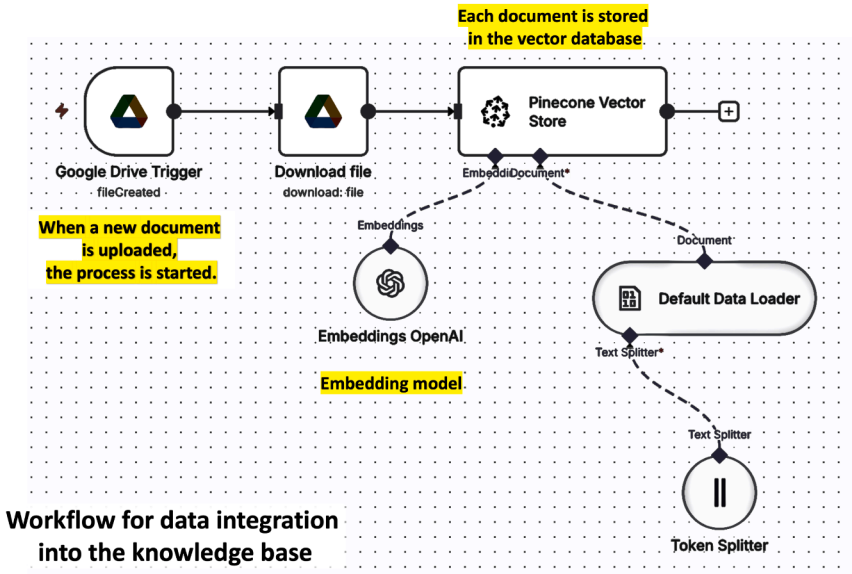
2.4. Agent-Based service layer (cloud or local)

Model training, evaluation, and computationally intensive analytics run either on-premises (GPU/IPC) or in the cloud. Representative functions include:

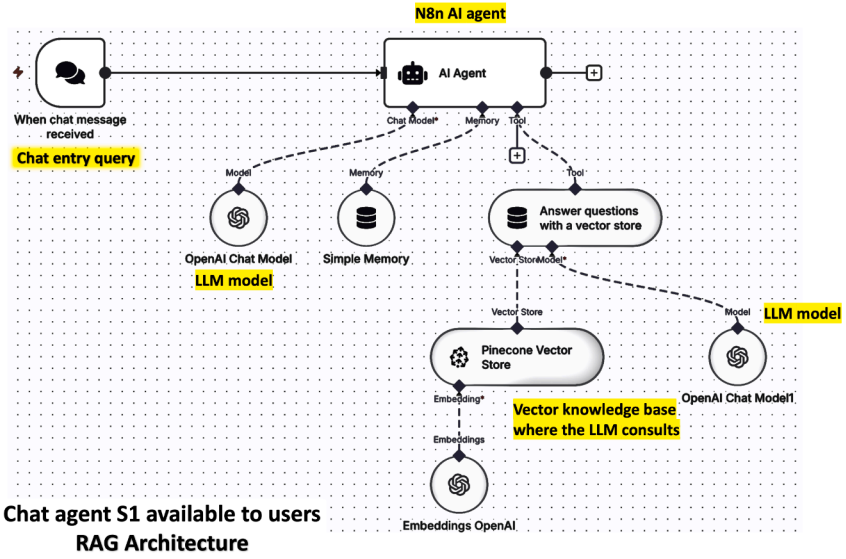
- **Forecasting and what-if analysis:** short-term forecasting of key variables, irrigation demands, and other resource needs.
- **Anomaly detection:** multivariate residual analysis, concept-drift alerts, and early-warning diagnostics.
- **Agent services:** the ASA service layer integrates multiple intelligent agents. The LLM-backed Chat-Agent (S1) is built on a RAG architecture, enabling the agent to retrieve and reason over recent operational metrics as well as stored Standard Operating Procedures (SOPs). This approach allows the system to incorporate the farmer's own knowledge and domain-specific know-how into decision support. The IoT-Agent (S2-IoT) is responsible for the distribution and enforcement of operational policies to field devices, ensuring that sensing, actuation, and process control remain aligned with predefined goals and adaptive strategies. Additionally, an offline reinforcement-learning (RL) framework with embedded safety filters is employed to refine decision-making policies from historical datasets, while preventing unsafe or undesirable actions in the production environment.
- **Data services:** time-series storage, feature stores, and dashboards that provide structured access to data and model outputs.

The ASA service layer can host a wide variety of intelligent services. The following items illustrate representative examples implemented or envisioned in this work:

- **S1 Chat-Agent** (Fig. 2): an LLM-backed conversational agent for planning, supervision, report generation, and guidance based on SOPs, integrating recent operational data with the farmer's own *know-how* through a RAG architecture.
- **S2-IoT agent** (Fig. 3): a telemetry-driven monitoring and actuation agent that continuously processes alerts and events, proposes or executes constrained actions (e.g., short irrigation cycles), and emits warning messages or other system interactions.
- **S2-E emergency agent** (Fig. 3): a specialized agent for singular events and extreme-condition scenarios, focused on anomaly



(a) Knowledge-base ingestion workflow. Documents (manuals, SOPs, records) are processed into embeddings and stored in Pinecone for semantic retrieval.



(b) Workflow for the Chat-Agent service. User or system queries trigger context retrieval from the Pinecone knowledge base, followed by response generation using the OpenAI LLM. The chat agent responds according to the retrieved knowledge.

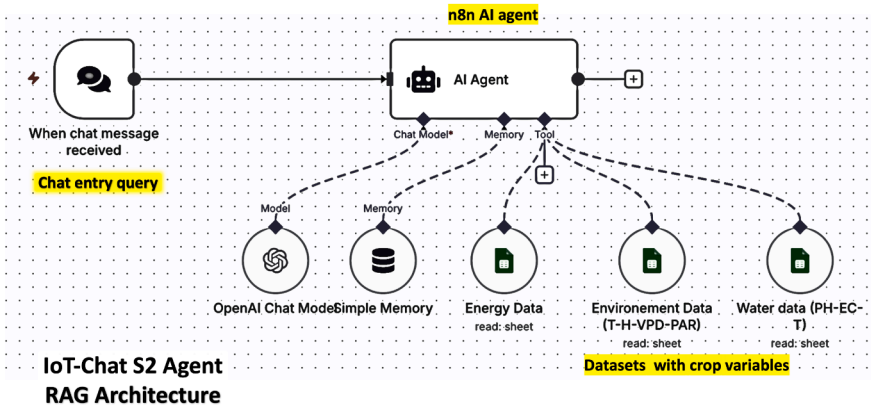
Fig. 2. S1 Chat-Agent. Implementation of a Retrieval-Augmented Generation (RAG) Chat-Agent deployed on n8n, integrating a Pinecone vector database with an OpenAI Large Language Model (LLM). The architecture consists of two workflows: one for ingesting and embedding domain documents into the vector database, and another for delivering context-aware chat responses by combining retrieved knowledge with LLM reasoning.

detection, alert generation, and constrained automated actuation in line with adaptive operational policies.

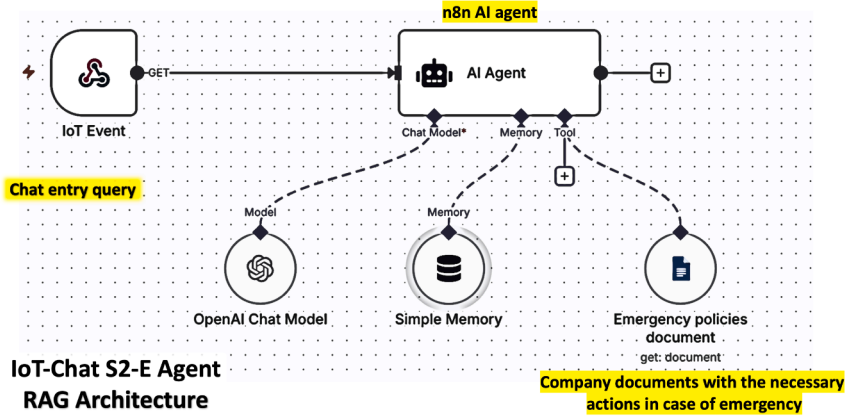
- **Analytics and forecasts**: services for generating yield proxies, key performance indicators (KPIs) for energy and water use, and predictive analytics to support medium- and long-term operational decisions.

2.4.1. *Implementation details of the AI components*

RAG Chat-Agent (S1). The ingestion workflow converts domain documents (e.g., SOPs, manuals, datasheets) into chunks with overlap, computes embeddings, and stores them in a vector database together with provenance metadata (document, section, timestamp). At query time, the retriever returns the top-k semantically closest chunks, which are injected into a structured prompt template. The Chat-Agent is configured to cite the retrieved sources, request missing context when needed, and avoid actuation; it produces guidance text and structured suggestions that can be audited. **Reinforcement learning (RL) component.** The architecture supports an RL policy module for setpoint optimization (e.g., irrigation/lighting scheduling) using greenhouse state features (T, RH, VPD, PAR/DLI, substrate or solution variables) and resource costs. Importantly, in the present industrial validation the RL module is *not used for direct actuation*; instead, control remains rule/PID-based under safety bounds, and RL is treated as an optional learning service to be activated once trained and validated (e.g., in simulation or shadow



(a) Chat-Agent for crop monitoring and grower interaction. This conversational agent reports crop evolution to the grower. Its knowledge base integrates historical and real-time telemetry from the greenhouse IoT infrastructure. The grower can query past and current datasets, request statistics, and obtain comparative analyses. Powered by an LLM, the agent interprets natural-language queries, processes relevant telemetry, and delivers actionable agronomic insights, improving situational awareness and decision-making.



(b) IoT-Agent for event-driven actuation. This agent is activated upon detection of emergency or predefined high-priority events within the installation. It operates on a knowledge base containing standard operating procedures (SOPs) for each event type. When triggered, the agent selects the relevant SOP, determines the prescribed actions, and executes them through the control layer, ensuring timely and consistent responses to critical situations.

Fig. 3. S2 IoT+E agent. Same implementation pattern as in Fig. 2, with two workflows and two types of IoT services deployed. Additional IoT services can be instantiated depending on the installation type or the processes to be automated.

mode) under the same policy gate described in Section 2.3.1. This clarifies which AI elements are implemented and evaluated versus provided as extensible modules.

2.5. User interfaces

Local users (operators) interact via mobile or console interfaces for routine tasks, acknowledgements, and direct control. **Cloud users** (managers, developers) access dashboards, reports, and model-management interfaces. The Chat-Agent provides natural-language question answering, explanations, and safe command templates; all commands require explicit confirmation and are bounded by predefined constraints.

2.6. End-to-End data flow (Fig. 4)

1. Sensors publish to HTTP/MQTT topics with timestamps and units; actuators subscribe on the same transport to control topics and accept validated command payloads (e.g., setpoints, on/off states, durations) issued by the orchestration and ASA layers, enabling closed-loop operation.
2. Basic local control and reactive actuation are executed using IoT communication protocols (e.g., MQTT/HTTP) and lightweight embedded algorithms on edge devices (rule-based triggers, debouncing, time guards).
3. In the orchestration layer, a local event engine evaluates policies and emits alerts or bounded commands. The controller API executes commands; actuators acknowledge them; and an audit log records all actions. Summaries and telemetry are forwarded to data services, where models are trained and updated.
4. In the ASA service layer, two main classes of agents are deployed: (S1) user-facing information services (Chat-Agent) that retrieve context from IoT telemetry, SOPs, and the vectorized knowledge base to answer queries and generate reports; and (S2) autonomous services that continuously ingest datasets, alerts, and enriched features to evaluate policies and then propose or execute constrained actions (e.g., short irrigation cycles), emit warning messages, or trigger other interactions with the system and with users. For sensitive interventions, an operator-in-the-loop approval step can be required before actuation.

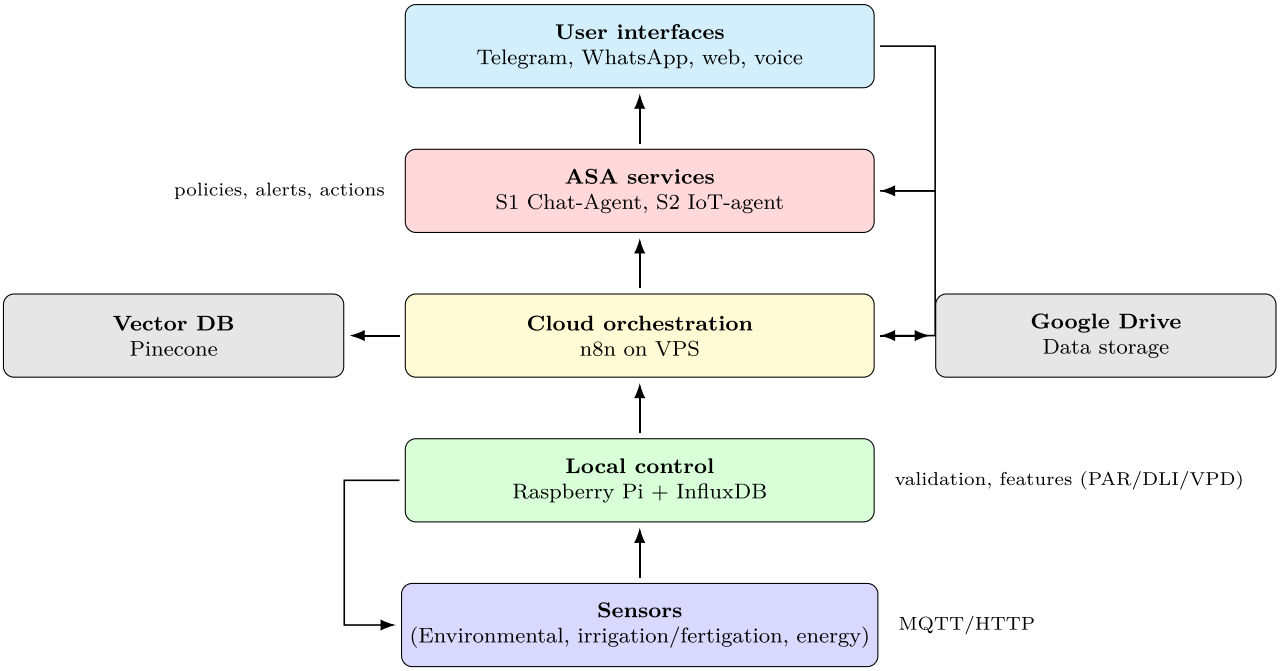


Fig. 4. ASA architecture and bottom-up data flow. From bottom to top: sensors → edge (Raspberry Pi + InfluxDB) → local/cloud orchestration (n8n on VPS) → ASA services (S1/S2) → user interfaces. Side integrations connect the orchestration layer with Pinecone (vector database) and Google Drive.

5. The platform exposes multichannel, user-friendly interfaces via Telegram and WhatsApp bots, a responsive web dashboard, and optional voice interaction. Messaging channels integrate through secure webhooks and bot APIs to the orchestration layer, enabling bidirectional workflows: growers can issue queries or commands and receive alerts, reports, and recommendations in real time. The web UI consolidates historical trends, KPIs, and agent outputs, while voice use leverages built-in speech-to-text and text-to-speech in the chosen platform for hands-free operation. All channels share a common identity and role-based access model (operator/agronomist/admin), with audit logging, rate limiting, and confirmation prompts for sensitive actions. Attachments (e.g., images, CSVs) can be ingested as context for the agents, and notifications are prioritized (info/warning/critical) to ensure timely escalation across devices.

2.7. LLM-Assisted actuation: risks and mitigations

We explicitly separate S1 (advisory), S2 (routine actuation), and S2-E (emergency). Before any command leaves the ASA layer, a *policy gate* validates constraints, provenance, and approvals:

- **Human-in-the-loop:** high-impact actions (e.g., pump state changes, setpoint shifts) require explicit operator confirmation, except under S2-E.
- **Safe ranges:** each actuator has hard bounds and maximum rate-of-change; out-of-range or abrupt changes are rejected.
- **Two-person rule (S2-E):** emergency actions proposed by the LLM must be corroborated by an independent ruleset or a second model; otherwise, the system degrades to S1 plus alarm.
- **Command contracts:** LLM outputs are parsed against a strict schema (`zone`, `action`, `value`, `ttl`); unsigned or ill-typed fields are rejected.
- **Shadow mode and rollback:** new policies and models initially run in shadow mode to compare against ground truth. If drift is detected, commands are blocked and prior setpoints are restored.
- **Kill switch and watchdog:** a hardware/software interlock can disable all S2 traffic. A watchdog reverts actuators to safe defaults on heartbeat loss or repeated policy failures.

2.7.1. Offline fallback and edge behavior under connectivity loss

Cloud connectivity cannot be assumed in operational greenhouses. In our deployment, the ESP-based controllers and the local Home Assistant (HA) hub operate in an *edge-first* manner: sensing, validation (V1), and the basic control loops described in Section 3.2 continue to run locally when the link to the cloud n8n orchestrator is unavailable. During outages, the system behaves in an edge-first fail-safe mode. (i) Local sensing, validation (V1), and the basic control loops continue to run on the ESP controllers and the Home Assistant hub. (ii) A connectivity watchdog raises an explicit OFFLINE state after a configurable timeout; this state is visible on the local dashboard and triggers local notifications (e.g., push notification / on-premise alerting) indicating that cloud orchestration is unavailable. (iii) Telemetry and events are persisted locally with timestamps, and a bounded local queue buffers messages for later synchronization. (iv) Safety is maintained by enforcing conservative local limits (minimum/maximum duty cycles, maximum irrigation pulses per window, and stage-dependent hard thresholds). When connectivity is restored, the system performs a controlled resynchronization: buffered events are uploaded and marked as historical (timestamped) to preserve auditability. If the backlog contains conditions that would have triggered an alarm while offline, the system generates a catch-up notification summarizing the offline interval and the abnormal episodes detected (including their timestamps). Importantly, the system does not retroactively actuate based on historical alarms; instead, it re-evaluates the current state immediately upon reconnection and triggers real-time alarms/actions only if the unsafe condition persists.

2.8. Limitations

Residual risk remains under correlated sensor faults or novel failure modes beyond the training and evaluation distribution. We mitigate this risk through redundancy, conservative defaults, and staged rollouts, but full elimination of risk is infeasible in open environments.

2.9. Design properties

The model is modular (layers can be swapped), portable (edge or cloud), and safety-oriented (explicit constraints and auditability). It

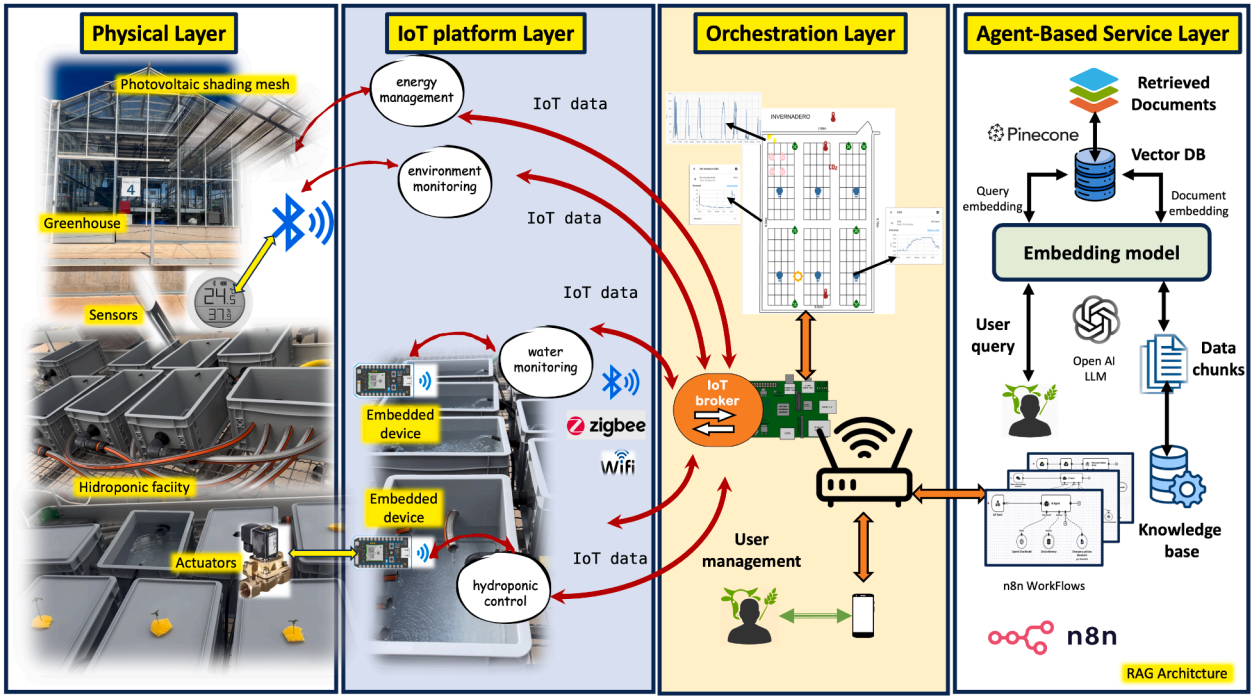


Fig. 5. Experimental deployment of the proposed four-layer architecture in a hydroponic greenhouse. The setup integrates physical infrastructure (sensors and actuators), an IoT platform for telemetry aggregation, orchestration via a local broker and Home Assistant, and a cloud-based agent-based service layer implementing RAG-enabled AI agents for user interaction, monitoring, and constrained actuation.

Table 2
Indicative cost drivers and trade-offs across deployment profiles. Values are illustrative and provider-dependent.

Profile	Typical components	Main trade-offs / cost drivers
Edge-only	Raspberry Pi/PC running HA + InfluxDB + dashboards; optional local ML inference	No recurring cloud fees; highest upfront integration effort for advanced AI; limited by local compute; simpler data governance.
Hybrid (this work)	Edge stack + low-cost VPS (n8n) + vector DB + managed LLM API	Modest recurring costs; fast integration via low-code; access to state-of-the-art LLMs; requires connectivity management + policy gate for safety.
Cloud-heavy	Thin edge gateway + cloud storage/compute; centralized analytics and control	Highest recurring fees; vendor lock-in risk; larger attack surface; may improve scalability for multi-site operations if managed well.

adheres to open, standardized protocols (e.g., MQTT, HTTP/REST, OPC UA, Modbus/TCP), explicitly avoiding proprietary lock-in. Wherever viable, services are implemented with low-code or no-code tooling and open-source components, reducing engineering effort and license costs. The architecture supports incremental adoption—from monitoring-only configurations to fully closed-loop control—without major refactoring. In sum, it is scalable and modular, adaptable to diverse agronomic installations, economically sustainable in deployment and maintenance, and backed by user-friendly interfaces that are simple to access.

2.10. Cost considerations and deployment trade-offs

Table 2 summarizes indicative cost drivers and trade-offs across three deployment profiles. Exact costs vary by provider and workload; therefore, we report the main *cost drivers* and operational implications rather than fixed prices. In small/medium deployments, the hybrid approach can reduce engineering effort (fewer custom integrations) and accelerate iteration, while edge-only deployments minimize recurring fees at the expense of AI capability and integration time.

2.11. Experimental setup (hydroponic greenhouse case study)

Industrial hemp was grown in a 50 m^2 glasshouse. Sensors included multi-zone air T/RH, lux/radiation, water temperature, PH/EC, and tank load cells. Telemetry frequency ranged from 1 min to 5 min. PAR, DLI, and VPD were computed online.

The experimental setup, as depicted in Fig. 5, materializes the proposed theoretical four-layer architecture in a real-world hydroponic greenhouse scenario.

Physical Layer: This layer is implemented through the actual infrastructure of the glass-covered hydroponic facility, comprising nutrient film technique (NFT) channels, pumps, and actuators for irrigation and nutrient dosing. Multi-modal sensors capture environmental (temperature, RH, PAR, PPFD, VPD, Lux, solar radiation, etc.) and hydroponic parameters (PH, EC, water temperature, tank weight). These sensors operate via a mix of wireless protocols (BLE, Zigbee, Wi-Fi) and wired connections to embedded devices based on ESP microcontrollers.

IoT Platform Layer: Embedded devices aggregate telemetry from the sensors and transmit it to the local IoT platform. This layer also handles specific monitoring domains-energy, environment, and water management-before forwarding structured IoT data streams. Communication between devices and the platform uses lightweight, low-power protocols for efficient in-situ deployment. **Orchestration Layer:** A local Raspberry Pi 4 running Home Assistant (HA) coordinates the integration of devices, data storage (InfluxDB), and dashboard visualization. It also implements reactive automation rules, orchestrates actuation commands, and forwards processed datasets (CSV format) to the Agent-Based Service Layer. The IoT broker within this layer routes telemetry

and commands between field devices, user management services, and higher-level applications.

Agent-Based Service Layer: The AI services are deployed in the cloud via an n8n-based low-code platform, connected to a Pinecone vector database populated with domain-specific manuals and operational data of the crop. The RAG (Retrieval-Augmented Generation) architecture enables LLM-powered agents to provide two main capabilities: Farmer-facing conversational assistance for planning, Standard Operation procedures (SOP) guidance, and progress reporting. Telemetry-driven monitoring, anomaly detection, and constrained actuation, including emergency overrides. This experimental deployment validates the feasibility of mapping the theoretical model into a functional, modular system that integrates IoT, orchestration, and AI-based services. Furthermore, it demonstrates that the proposed architecture can be adapted to other agronomic contexts beyond hydroponics by altering the Physical Layer while keeping upper-layer services intact.

While the ASA model described in the previous sections is presented as a theoretical framework applicable to a wide range of agronomic installations, this section illustrates its practical implementation in a specific context: an industrial-hemp hydroponic crop within a controlled-environment greenhouse.

Once the AI agents were deployed, the grower interacted with the **Chat-Agent** through two main functionalities, accessible via user-friendly interfaces and widely adopted digital platforms such as Telegram, WhatsApp, and web access from mobile devices, tablets, or desktop computers. These interactions also leveraged the built-in voice recognition services of these platforms, enabling hands-free operation and improving accessibility for different user profiles:

- **Consultation and planning:** Acting as a virtual assistant, the Chat-Agent responded to questions about crop planning, nutrient requirements, and desired environmental conditions. It was trained with domain knowledge of the crop to provide informed and context-aware guidance from the outset of the process.
- **Status and progress reporting:** Leveraging data captured by the IoT layer and processed through the lower layers of the ASA, the Chat-Agent generated and delivered periodic reports summarizing crop status, environmental trends, and recommended actions. This ensured that the grower remained continuously informed of real-time conditions and the evolution of the cultivation process.

In addition, the **IoT-Agent** demonstrated its potential to:

- Interact directly with the control system following predefined operational rules or adaptive policy sets.
- React automatically to events that require immediate intervention, such as extreme environmental deviations or critical system faults.
- Generate and dispatch alerts to users, enabling prompt human oversight or intervention when necessary.

3. Results

The facility consists of a 50 m^2 glass-covered greenhouse equipped with:

- A recirculating hydroponic system with nutrient film technique (NFT) channels, developed within the European project Purecircles [31].
- Environmental sensors measuring temperature, relative humidity, CO_2, PAR, VPD, solar radiation, illuminance (lux), and related variables.
- Nutrient solution monitoring (pH and EC).
- A photovoltaic (PV) shading mesh for solar-radiation control and on-site energy production (SolarCloth)—a pilot installation currently under validation in our greenhouse (Fig. 6) [32].
- IoT-enabled on/off actuators for water tank, ventilation, irrigation, and shading control.

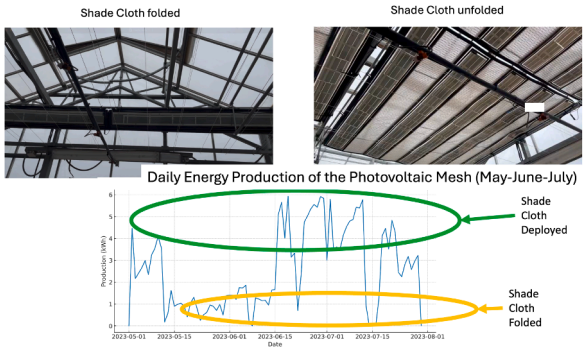


Fig. 6. Energy production from the solar shade cloth and associated sensor data.

The IoT device-network layer comprises sensors connected via wireless BLE interfaces to ESP-based microcontrollers, forming a distributed sensor network, as well as sensors that connect directly to the local Wi-Fi network deployed in the facility. Actuator signals are transmitted through these same communication channels. The local Wi-Fi network is generated by a router with Internet connectivity, enabling both local and remote access to the system.

A Raspberry Pi 4 running the Home Assistant operating system serves as the core IoT hub, allowing the implementation of reactive automation rules, data storage, and the development of real-time supervision and monitoring dashboards. This platform not only sends processed telemetry to the AI agents but can also execute commands or requests originating from them, thus closing the loop between sensing, decision-making, and actuation.

The crop trial was intentionally conducted without direct intervention on environmental parameters—such as temperature, humidity, or solar radiation—in order to evaluate the baseline performance and resilience of the hydroponic configuration under naturally fluctuating climatic conditions. Environmental, fertigation, and energy sensors were strategically distributed throughout the greenhouse to ensure multi-zone coverage and accurate representation of microclimatic variations. Cultivation outcomes, including biomass accumulation and visual crop status, are presented in Fig. 7. Following the installation and commissioning of the first three layers of the proposed Agent-Based Service Architecture (ASA) model, the workflows corresponding to the intelligent agents described in the previous section were deployed. The Chat-Agent facilitated bidirectional interaction with the grower, while the IoT-Agent continuously monitored telemetry, generated alerts, and executed constrained actuation commands when necessary.

The greenhouse IoT stack is connected to the orchestration and ASA service layer via MQTT and HTTP endpoints, enabling:

- Data ingestion from environmental and fertigation sensors. Real-time data are stored and pre-processed in the orchestration layer. At user-scheduled times, summarized datasets are sent to the ASA layer.
- Execution of S1 (Chat-Agent) queries for crop status, SOP guidance, and growth planning.
- Continuous monitoring, anomaly detection, and constrained actuation by the S2-IoT-Agent.
- Triggering of the S2-E-Agent (emergency actuation) in extreme scenarios, e.g., pump failure or high-heat events. In such cases, the relevant data are immediately sent to the ASA layer.

ASA Service Catalogue

Beyond describing the architectural layers, it is useful to characterize the concrete services exposed by the ASA in the deployed greenhouse. Table 3 summarizes the main ASA services, detailing their inputs, outputs, interaction patterns, and representative performance indicators.

Fig. 7. Evolution of the crop used in the proof-of-concept trial.

Table 3
Catalogue of ASA services deployed in the greenhouse use case.

Service	Role / type	Main inputs	Main outputs	Trigger / interaction pattern	Representative KPIs
Chat-Agent (S1)	User-facing conversational agent for crop consultation, planning, and report generation	Natural-language queries from growers; vectorized knowledge base containing crop documentation, SOPs, and IoT-derived summaries	Natural-language answers; structured reports (text + JSON) including statistics, threshold checks, and recommended actions	User-initiated via messaging apps (Telegram, WhatsApp) and web dashboards; periodic, scheduled reporting (daily / weekly)	Question-level exact match and tolerance-based scores; semantic-similarity metrics; response time per query
S2-IoT-Agent	Telemetry-driven monitoring and constrained actuation agent	Time-series IoT data exported from the Home Assistant / InfluxDB stack; contextual metadata (crop stage, setpoints, sensor locations)	Aggregated statistics; anomaly flags; enriched events; constrained actuation commands forwarded to the control platform	Periodic execution over predefined datasets; event-triggered analysis when new IoT batches are available or thresholds are approached	Numeric accuracy (MAE, RMSE, MAPE, R^2); anomaly-detection precision/recall; end-to-end analysis latency
S2-E-Agent	Emergency actuation and safety-policy agent	Real-time telemetry from the IoT broker; safety-policy knowledge base with stage-dependent thresholds for environmental and fertigation variables	JSON alarms; recommended or automatic corrective actions; escalation messages to operators or supervisor dashboards	Event-driven: executed whenever incoming measurements violate policy thresholds or when fault events are detected	Time to alarm; proportion of correctly detected unsafe events (true positives); rate of false alarms (false positives); service availability
Orchestration workflows (n8n)	Low-code orchestration layer coordinating IoT platform, ASA services, and external UIs	MQTT messages and HTTP webhooks from Home Assistant; scheduled triggers; user commands; API keys and routing rules	Service invocations (LLM calls, database queries); notifications to messaging platforms; actuation commands back to Home Assistant	Mix of cron-based workflows (e.g., periodic reporting) and event-driven flows (sensor updates, emergencies, user requests)	End-to-end workflow latency; success/failure rate per workflow; executions per day; resource usage on the orchestration VPS

This catalogue clarifies how the Chat-Agent (S1), the IoT monitoring agent (S2-IoT-Agent), the emergency agent (S2-E-Agent), and the orchestration workflows cooperate to close the loop between sensing, reasoning, and actuation in the installation.

Orchestration Workflows as Service-Level Experiments

In addition to the individual services, the ASA relies on low-code workflows that orchestrate IoT telemetry, ASA agents, and user interfaces. From an experimental perspective, these workflows can be regarded as service-level pipelines whose behavior can be characterized in terms of latency, robustness, and execution patterns. Table 4 summarizes a set of representative workflows deployed during the trial, illustrating how events originating in the IoT layer are transformed into AI-enhanced decisions and, when appropriate, into automatic or operator-approved actions.

In this implementation, an `n8n` orchestration platform was deployed on a cloud-based virtual private server (VPS), although the same setup could be hosted locally. Data storage is handled through a local `InfluxDB` instance running on the Raspberry Pi, complemented by periodic exports to Google Drive files to facilitate AI-agent interaction.

A dedicated vector database was pre-populated in `Pinecone` with crop-specific documentation, operating procedures, and technical manuals for the selected cultivation type. This knowledge base is queried by the agents to generate context-aware outputs and recommendations.

All employed platforms are low-code compatible and offer free or low-cost versions, enabling affordable deployment while maintaining high flexibility and scalability in service orchestration.

From the perspective of the ASA layered model, this integration bridges the lower layers—comprising sensing, local data processing, and control—with the upper layers of orchestration and intelligent services. The IoT devices and local controller (Raspberry Pi) handle data acquisition and initial processing, while the cloud-based orchestration platform coordinates service execution and AI-agent interactions. This multi-layer interaction ensures a seamless data flow from field sensors to decision-making agents and, when necessary, to automated or operator-approved actuation, thus closing the loop between monitoring, reasoning, and action in the agronomic installation.

Table 4
Representative orchestration workflows connecting the IoT platform with ASA services.

Workflow	Description	Trigger pattern	Involved ASA services	Monitored metrics
W1 – Environmental safety monitoring	Real-time evaluation of temperature, RH, CO_2 and related variables against stage-dependent safety policies. Violations generate alarms and corrective actions through the control system.	Event-driven: MQTT/HTTP messages from the IoT hub whenever new sensor readings are available.	S2-E-Agent (safety policy enforcement); S2-IoT-Agent (contextual enrichment); orchestration layer for routing and logging.	DNS lookup, connection setup, TTFB, and total response time (as in Fig. 14); proportion of correctly handled unsafe events; alarm-delivery success rate.
W2 – Daily crop-status report	Generation of a daily natural-language summary of crop status, environmental trends, and recommended actions, delivered to the grower via messaging platforms.	Time-based: cron-like trigger at predefined times (e.g., once per day in the evening).	S1 Chat-Agent; S2-IoT-Agent for statistical summaries; vector database for domain knowledge; messaging connectors (Telegram / WhatsApp).	End-to-end latency from scheduled trigger to report delivery; successful-delivery rate; average number of tokens per report; user-perceived usefulness (questionnaire or informal feedback).
W3 – Batch IoT dataset analysis	Periodic analysis of IoT datasets exported from the greenhouse (e.g., daily or weekly CSV files) to compute descriptive statistics, anomalies, and consumption indicators. Results are stored and optionally summarized by S1.	Hybrid: time-based for dataset export plus user-initiated execution when new files are uploaded.	S2-IoT-Agent (numeric analysis); S1 Chat-Agent (natural-language explanation); storage services for results.	Numeric accuracy metrics (MAE, RMSE, MAPE, R^2) with respect to ground-truth calculations; processing time per dataset; number of detected anomalies.
W4 – Emergency escalation	Escalation of critical alarms (e.g., pump failures, extreme temperatures) to human operators through high-priority channels and, if necessary, redundant notifications.	Event-driven: activation when S2-E-Agent emits a high-severity alarm.	S2-E-Agent; orchestration layer for channel selection and redundancy; external notification services (e-mail, messaging).	Time from alarm generation to first notification; success rate of notification delivery; proportion of alarms acknowledged by operators within a defined time window.

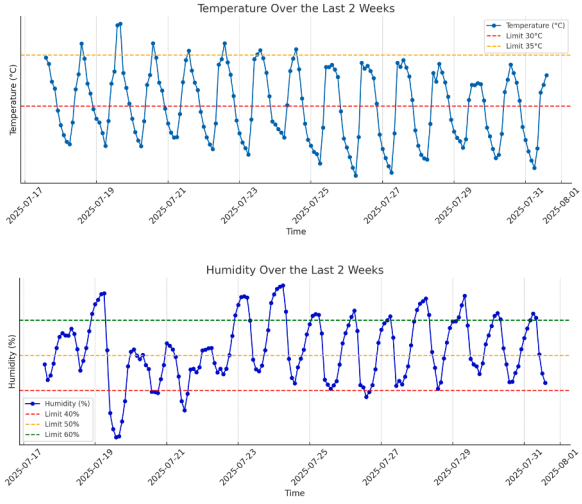


Fig. 8. IoT data validation.

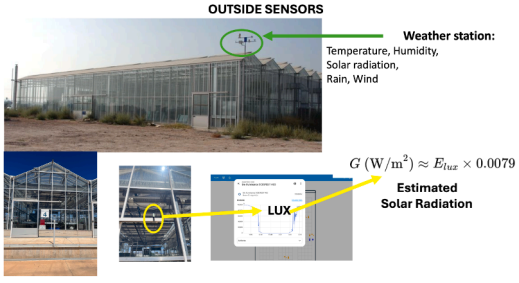


Fig. 9. IoT weather station.

3.1. Platform and intelligent-Agent validation

The validation process focused on demonstrating the end-to-end operation of the proposed architecture. Instead of measuring sensor accuracy, the evaluation ensured that data flows were correctly captured, stored, processed, and made available to both intelligent agents and end users through the dashboards and interfaces.

3.1.1. IoT Platform validation

The IoT layer was validated by deploying a heterogeneous set of sensors and actuators in the greenhouse. The validation aimed to confirm:

- **Data acquisition:** sensor streams (environmental, fertigation, and energy-related) were successfully collected at predefined frequencies (1–5 minutes) (Fig. 8).
- **Data persistence:** all telemetry was reliably stored in the local InfluxDB database and mirrored to Google Drive files, ensuring redundancy and external access.
- **Data communication:** periodic datasets were correctly transmitted to the higher layers, where they were available for further processing by the AI agents.

The results confirmed that the IoT platform maintained continuous operation during the proof-of-concept period, providing uninterrupted data streams to the upper service layers.

3.1.2. User interface validation

The validation of user interfaces focused on ensuring that the information collected by the IoT platform and processed by ASA agents was accessible and understandable to growers. Tests included:

- **Dashboards:** the Home Assistant platform successfully displayed real-time crop and environmental conditions, with historical trends available for comparison (Fig. 9, 10, 11, and 12).
- **Cross-platform access:** users accessed the dashboards through mobile devices, tablets, and web browsers without requiring additional configuration.
- **Messaging tools:** integration with platforms such as Telegram and WhatsApp enabled conversational interaction with the Chat-Agent, including both queries and report delivery.

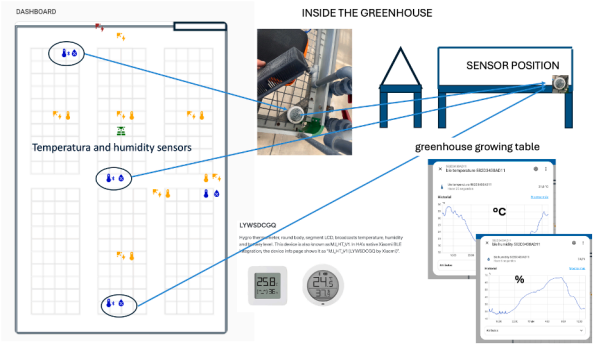


Fig. 10. Environmental sensors and dashboard.

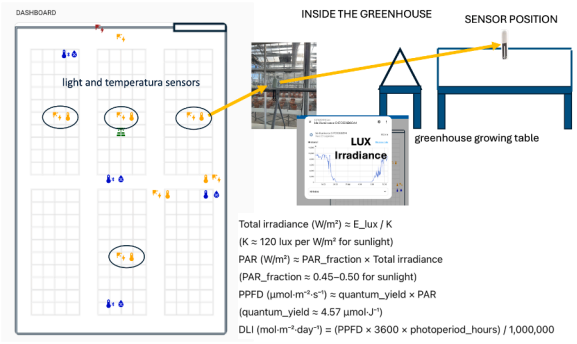


Fig. 11. Illuminance sensors and dashboard.

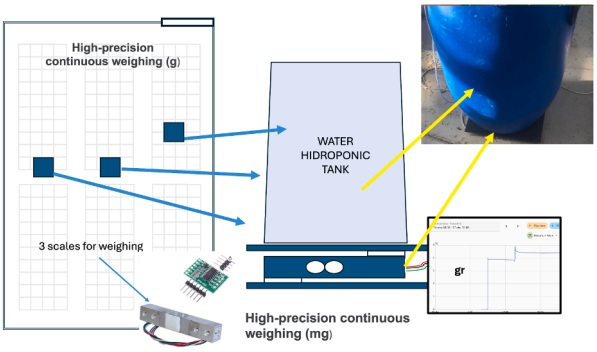


Fig. 12. Weight sensors and dashboard.

These validations confirmed that the system provided growers with continuous, user-friendly access to crop information.

3.1.3. Validation of intelligent agents (S1 and S2)

The validation of the intelligent agents focused on verifying their capacity to process IoT telemetry, support user interaction, and trigger relevant actions. Two aspects were evaluated: (i) the Chat-Agent (S1) was tested for consultation, reporting, and planning tasks, while (ii) the IoT-Agent (S2-IoT-Agent) was validated under normal operation, anomaly detection, and simulated emergency events. Both agents successfully ingested data from the IoT layer and provided outputs through user-friendly dashboards and conversational interfaces.

An additional outcome from validation highlights the importance of establishing a structured *prompt-planning strategy* for the Chat-Agent. By systematically designing prompts around reports, statistical analyses, and historical comparisons, the agent can deliver more consistent outputs and unify agronomic insights across different cultivation cycles. This enables growers not only to monitor the current crop in real time but also to perform comparative analyses with past and future cultivation campaigns, strengthening data-driven decision-making.

3.1.3.1. S1 Chat-Agent validation.. Using the knowledge base built from IoT data and cultivation manuals, the Chat-Agent was tested for two main functionalities: (i) consultation and planning (nutrient scheduling, environmental targets, cultivation strategies), and (ii) reporting (summaries of crop status and evolution). Both functionalities were successfully demonstrated (Fig. 13).

S1 question typology in greenhouse cultivation. In order to evaluate the performance of the Retrieval-Augmented Generation (RAG) agent implemented within the ASA architecture, it is essential to classify the types of questions that users typically formulate in the domain of greenhouse cultivation. Different question types correspond to different expected answer formats and therefore require tailored evaluation strategies. Table 5 summarizes the main categories of questions identified, along with representative examples and the corresponding evaluation metrics.

The proposed taxonomy suggests that the RAG system applied in greenhouse cultivation should incorporate a preliminary stage that automatically classifies the user query according to its type. Once the question type is identified, the query can be redirected to a specialized submodule optimized for handling that specific category, whether it involves numeric values, procedural knowledge, causal reasoning, or regulatory information. This modular approach not only improves the accuracy of the retrieved and generated answers but also facilitates the application of the most suitable evaluation metrics for each case. For each type of question, different databases may be consulted. It is not the same to ask a question about a task, where multiple options without major consequences may be acceptable, as to ask for a numeric value that must be explicitly stated in a security-policy document.

Table 6 presents a subset of representative questions posed by the grower and the corresponding responses generated by the Chat-Agent. To complement the qualitative assessment, we report ROUGE-L, BLEU-4 and BERTScore over the 40 Q&A items, grouped into **True**, **Erroneus**, and **Not found** categories (Table 7) analyzed in Section 4 The S1 evaluation set was designed as a coverage-oriented benchmark, not as a population survey. We grounded the selection in the nine greenhouse question types summarized in Table 5 (numeric, range/threshold, causal/effect, procedural, categorical/definition, temporal/schedule, diagnostic, optimization, and regulation/quality). The final set includes multiple items per category to ensure that both retrieval and generation are exercised across heterogeneous answer formats, while keeping expert labeling feasible and reproducible. With $n = 40$, the estimated accuracy can be reported with a reasonably tight uncertainty band; for example, 38/40 correct corresponds to a 95% Wilson confidence interval of approximately 0.835–0.986, which is sufficiently informative for model comparison under identical RAG conditions. We also clarify that expanding the benchmark further is straightforward and is planned as part of multi-cycle and multi-site future validation.

3.1.3.2. S2-IoT-Agent validation.. This agent was validated by processing periodic datasets generated by the IoT platform. It enriched events, detected anomalies, and issued alerts to the operator. In addition, different LLM backends were tested: GPT-4 models were able to capture context correctly but presented errors in basic arithmetic and statistical calculations, whereas GPT-5 eliminated these issues and produced consistent, accurate results. In controlled scenarios, the agent triggered safe actuation commands (short irrigation cycles) via the control API, confirming its ability to close the loop between monitoring and action. The conversational workflow used for IoT data validation is illustrated in Fig. 13. A latency and functional benchmark of the S2-IoT-Agent is presented in Section 3.3.

Query types for IoT data analysis. In this work, we developed and tested an automated question-answering system for IoT datasets stored in spreadsheets. The system is capable of interpreting natural-language queries and providing structured JSON responses. Different categories of questions were supported, each corresponding to a type of data analysis frequently required in precision agriculture and IoT-based monitoring.

Table 5
Global classification of question types, examples, and recommended evaluation metrics for RAG in greenhouse cultivation.

Question Type	Description	Examples	Evaluation Metric
Numeric value	Expect a single numeric answer or measurement.	What is the optimal germination temperature for hemp seeds? What pH level is recommended for the nutrient solution?	Exact match, tolerance-based numeric match
Range / threshold	Expect an interval or acceptable limit.	What relative humidity range is recommended during flowering? What CO_2 concentration is considered too high?	Range matching, tolerance-based accuracy
Causal / effect	Address cause–effect relations in plant growth or environment.	How does excessive irrigation affect root development? What is the effect of high light intensity on biomass production?	Semantic similarity (BERTScore, BLEURT), effect classification
Procedural / steps	Require a sequence of actions or a methodology.	How should the substrate be prepared before transplanting? What are the steps of integrated pest management?	ROUGE-L, BLEURT, sequence-completeness check
Categorical / definition	Ask for classification or definitions of terms.	What is a Venlo greenhouse? What is the difference between drip irrigation and fertigation?	Semantic similarity (cosine embeddings), BERTScore
Temporal / schedule	Concern durations or timing of processes.	How many weeks does the vegetative phase last? In which month is it recommended to sow in Alicante?	Numeric match, normalized temporal expressions
Diagnostic / troubleshooting	Seek identification of problems or symptoms.	Why are the leaves turning yellow? What does mold on the stem indicate?	Semantic similarity (BLEURT, embeddings), LLM-as-a-judge
Optimization	Request strategies to maximize yield or efficiency.	What planting density optimizes light capture? What nutrient combination increases yield efficiency?	LLM-as-a-judge, BLEURT, expert-based evaluation
Regulation / quality	Related to standards, certification, or quality control.	What GMP requirements apply to medical cannabis cultivation? What quality parameters are checked at harvest?	Semantic similarity, Hit@k (retrieval relevance), LLM-as-a-judge

Table 6
S1 Chat-Agent evaluation question set ($n = 40$) and manual outcome labels based on the GPT-5.2 answer. Outcomes are **True** (correct), **Erroneous** (incorrect), or **Not found** (missing/insufficient retrieved context in the RAG knowledge base).

#	Question	Outcome	#	Question	Outcome
1	What does photoperiod mean in cannabis cultivation?	True	2	What is the optimal temperature for cannabis in the vegetative stage?	True
3	What pH is recommended for watering in soil?	True	4	What pH is recommended for watering in hydroponics?	Erroneous
5	What does EC mean in cannabis cultivation?	True	6	How many hours of light are recommended in the vegetative stage?	True
7	How many hours of light are recommended in the flowering stage?	True	8	What is “stretch” in cannabis?	True
9	Which nutrient is key in the vegetative stage?	True	10	Which nutrients are key in the flowering stage?	True
11	What is flushing or root washing?	True	12	What is pollination in cannabis?	True
13	What is the goal of growing cannabis without pollination?	True	14	What is a cutting or clone?	True
15	What is the advantage of clones?	True	16	What is VPD (Vapor Pressure Deficit)?	Not found
17	What is DLI (Daily Light Integral)?	True	18	How do you identify nitrogen deficiency?	True
19	How do you identify magnesium deficiency?	True	20	What are the symptoms of spider mite infestation?	True
21	What is the function of trichomes?	True	22	When is the optimal time to harvest based on trichome color?	True
23	What allows for sinsemilla production?	True	24	What is the Screen of Green (SCROG) technique?	True
25	What involves Low Stress Training (LST)?	True	26	How does high humidity affect flowering plants?	True
27	What is the risk of Botrytis (Bud Rot)?	True	28	What is the main advantage of hydroponics over soil?	True
29	What is an aquaponic system?	True	30	Why is dark period interruption harmful in flowering?	True
31	What is the recommended CO2 concentration for high-yield setups?	True	32	What is supercropping?	True
33	What is a hermaphrodite plant?	True	34	What causes hermaphroditism in cannabis?	True
35	How long should the curing process last?	True	36	What is the ideal humidity for drying cannabis?	True
37	What does PPM measure in nutrient solutions?	True	38	What is the ’Sea of Green’ (SOG) method?	True
39	What is the difference between SATIVA and INDICA growth patterns?	True	40	What is the role of Mycorrhizae?	True
…					

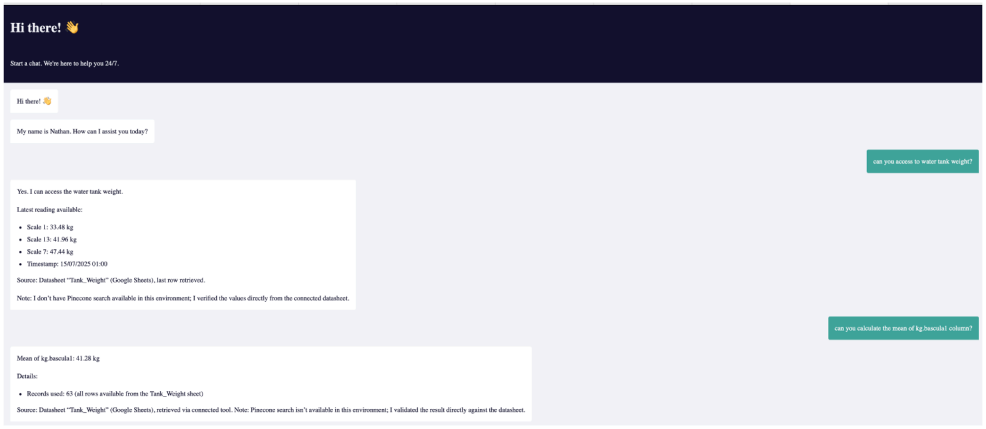


Fig. 13. Web-service access for S2-IoT-Agent validation. IoT datasets are served through a web interface and queried by the LLM-based agent. A vectorized knowledge base supports contextual analysis and generation of insights.

Table 7
Quantitative NLP metrics for the S1 Chat-Agent ($n = 40$) computed against reference answers. We report ROUGE-L, BLEU-4, and BERTScore (F1) for GPT-4o-Mini and GPT-5.2, both overall and by outcome subset (**V**: True, **E**: Erroneous, **NF**: Not found).

Model	Subset	n	ROUGE-L	BLEU-4	BERTScore
GPT-4o-Mini	Overall	40	0.198	3.410	0.874
GPT-4o-Mini	True	32	0.215	3.801	0.880
GPT-4o-Mini	Err.	3	0.146	1.996	0.854
GPT-4o-Mini	Not F.	5	0.118	1.755	0.851
GPT-5.2	Overall	40	0.272	3.821	0.872
GPT-5.2	True	38	0.269	3.877	0.872
GPT-5.2	Err.	1	0.467	3.490	0.893
GPT-5.2	Not F.	1	0.185	2.045	0.826

Advantages of S2-IoT-Agent data querying. Traditional IoT monitoring and data-management platforms usually provide a set of predefined queries or fixed dashboards. While these interfaces are useful for routine supervision, they are inherently limited: the user can only access the information that has been explicitly configured by the system designer.

In contrast, the proposed agent-based approach allows the user to formulate **open-ended queries in natural language**, which are dynamically interpreted and resolved against the IoT datasets. This capability transforms the way data are explored, enabling flexible, on-demand analytics without the need for prior configuration of specific reports or query templates. Examples include descriptive statistics, threshold-based detections, range filtering, daily aggregations, refill-event identification, combined conditions, extremes by date, and comparative analyses between sensors or tanks.

By combining IoT data streams with an intelligent agent, the system extends beyond the traditional boundaries of static dashboards and provides a more powerful and adaptive decision-support tool. This paradigm democratizes data access, since end users (e.g., farmers, technicians, or researchers) can request any type of analysis in plain language and receive consistent, structured responses in real time.

3.1.4. S2-IoT-Agent Experiment: Solar radiation and DLI from lux sensors

In order to evaluate the analytical capabilities of the S2-IoT-Agent beyond emergency detection, a second experiment was designed around the estimation of solar radiation and light availability in the greenhouse. The aim was to assess whether the agent can transform low-cost illuminance measurements (lux) into agronomic light variables and produce daily reports comparable to expert assessments.

The experimental setup uses a ceiling-mounted lux sensor inside the greenhouse, configured with a sampling interval between 1 and 5 minutes. Each record contains the timestamp, the current crop stage (`Vegetative`, `Flowering`, `Harvest`), and the illuminance reading in lux. The data are exported as CSV files with the structure `timestamp, stage, lux`. From these time series, the S2-IoT-Agent is requested to estimate the photosynthetic photon flux density (PPFD, in $\mu mol\ m^{-2}\ s^{-1}$) and the daily light integral (DLI, in $mol\ m^{-2}\ d^{-1}$).

Since lux is a photometric quantity and PPFD is a radiometric quantity in the PAR band (400–700 nm), an approximate spectral conversion must be used. For greenhouse conditions with a daylight-like spectrum, a commonly used rule-of-thumb in horticulture assumes a proportionality between illuminance and PPFD of the form:

$$\mathrm{PPFD}\ [\mu mol\ m^{-2}\ s^{-1}] \approx \frac{\mathrm{lux}}{k},$$

where the factor k depends on the light spectrum. In this work, $k = 54$ is adopted as a representative value for natural and greenhouse-filtered sunlight, following a common horticultural rule-of-thumb for daylight-like spectra [33]. with the explicit caveat that this conversion introduces an approximation error that is acceptable for the purposes of decision support but not for precision radiometry.

Given the estimated PPFD time series, the S2-IoT-Agent computes the daily light integral (DLI) as:

$$DLI = \frac{1}{10^6} \sum_i \mathrm{PPFD}_i \cdot \Delta t_i \quad [\mathrm{mol}\, m^{-2}\, d^{-1}],$$

where PPFD_i is the instantaneous PPFD at sample i and Δt_i is the time interval (in seconds) between consecutive samples. For a constant sampling interval of 60 s, this expression reduces to:

$$DLI = \frac{60}{10^6} \sum_i \mathrm{PPFD}_i.$$

For each day and crop stage, expert-defined target ranges for DLI are specified (e.g., a lower DLI range in vegetative growth and a higher target range in flowering). Based on the computed DLI and the current stage, the S2-IoT-Agent is instructed to classify each day into one of three light-availability classes: `LIGHT_LOW`, `LIGHT_OK`, or `LIGHT_HIGH`. In addition, the agent generates a short natural-language summary describing the light conditions (e.g., "DLI below the target range due to overcast conditions") and simple agronomic recommendations (e.g., adjusting shading or considering supplemental lighting).

To obtain a ground truth for evaluation, the same lux time series are processed in a Python/Colab notebook, where PPFD and DLI are computed using the same conversion factor and formulas, and each day is labelled by an expert as `expected_light_class` $\in \{\mathrm{LIGHT\_LOW}, \mathrm{LIGHT\_OK}, \mathrm{LIGHT\_HIGH}\}$ according to the target DLI ranges for the corresponding stage. The S2-IoT-Agent is then queried via an HTTPS `POST` call to its webhook endpoint, and its predicted label `predicted_light_class` is recorded together with the estimated DLI and the generated textual report.

The quantitative evaluation comprises two parts. First, numerical accuracy of the DLI estimation is assessed by comparing the DLI values reported by the agent with those obtained in the Python notebook, using mean absolute error (MAE), root mean square error (RMSE), and relative error per day. Second, the classification performance on light availability is analysed by computing the overall accuracy:

$$\mathrm{Accuracy}_{\mathrm{light}} = \frac{\#\{d : \mathrm{predicted\_light\_class}_d = \mathrm{expected\_light\_class}_d\}}{N_d},$$

where N_d is the number of evaluated days, together with a confusion matrix over the three light classes. This setup allows us to quantify both the numerical quality of the radiometric approximation from lux readings and the practical usefulness of the agent's daily light reports for growers.

3.2. Evaluation of the S2-Iot agent using real greenhouse data

To assess the performance of the S2-IoT agent under realistic operating conditions, we selected a test set of daily cycles derived from real greenhouse sensor data (lux measurements sampled every 5 minutes). For each day, a reference daily light integral ($\mathrm{DLI}_{\mathrm{ref}}$) was computed in Python using the same lux-to-PPFD conversion and integration step as in the agent. Stage-specific agronomic thresholds were then applied to assign a ground-truth light class (LOW/OK/HIGH) to each day.

The performance metrics summarized in Table 8 report the numerical agreement between the S2-IoT agent and the reference DLI computation, as well as the overall accuracy of the light-class detection. In addition, Table 9 presents the confusion matrix for the three light classes, showing how often the agent predictions match the expected labels for each category. These results provide a quantitative validation of the S2-IoT agent as a reliable component for automated radiation monitoring and decision support in greenhouse cannabis production.

The purpose of this experiment is to validate the end-to-end telemetry pipeline (data export →retrieval →agent computation →reporting) and the correctness of the implemented radiometric approximation and integration logic under real operating conditions, rather than to claim seasonal generalization of irradiance patterns. The 13-day window provides 13 independent daily integration targets while still containing

Table 8
Performance of the S2-IoT agent for DLI estimation and light-class detection on the evaluation set.

Metric	Value	Description
MAE (mol m^{-2} d^{-1})	0.005	Mean absolute error between DLI_{ref} and DLI_{agent}
RMSE (mol m^{-2} d^{-1})	0.008	Root mean square error between DLI_{ref} and DLI_{agent}
Light-class accuracy	1.00	Proportion of correctly classified days (LOW/OK/HIGH)
Number of evaluated days	13	Days with valid reference and agent outputs

Table 9
Confusion matrix for light-class detection (LOW/OK/HIGH) by the S2-IoT agent. Rows correspond to expected labels, columns to predicted labels.

Expected	LIGHT_LOW	LIGHT_OK	LIGHT_HIGH
LIGHT_LOW	5	0	0
LIGHT_OK	0	4	0
LIGHT_HIGH	0	0	4

thousands of underlying lux samples (5-minute sampling) and covering realistic day-to-day variability (clear vs. overcast conditions and stage-dependent target ranges). Numerically, the agent reproduces the reference computation with very low error (Table 8), indicating that the computation and reporting chain is stable. For the categorical light-class output, 13/13 correct implies a conservative 95% Wilson lower bound of approximately 0.77, which is acceptable for a pipeline validation study and motivates our next step: extending across longer time spans and additional deployments to capture broader seasonal regimes (sun angle, weather distributions) and strengthen external validity.

In addition to reproducing DLI and light-class calculations, the S2-IoT agent includes a fail-safe mechanism that detects invalid sensor readings (e.g., NaN values or missing data). For these days, the agent does not emit a DLI value and instead raises an `ALARM` status with a human-readable explanation and recommended actions.

3.2.1. S2-E-Agent Validation

To ensure system robustness under extreme or abnormal conditions, a dedicated emergency agent (S2-E-Agent) was validated. This agent operates on top of the IoT monitoring framework and is responsible for reacting promptly to critical events that may compromise crop health, safety, or system integrity. Unlike the standard S2-IoT-Agent, which handles routine anomaly detection and constrained actuation, the S2-E-Agent enforces stricter policies with predefined escalation protocols.

Each policy specifies a triggering condition, typically derived from environmental or operational thresholds, and the corresponding corrective action to be executed automatically or escalated to the operator. These scenarios include high or low temperature events, irrigation pump failures, nutrient imbalances, reservoir depletion, and sensor malfunctions. The adoption of explicit policies allowed controlled validation of the S2-E-Agent, ensuring safe and reliable responses to emergency conditions during greenhouse operation.

S2-E-Agent practical examples. Several scenarios illustrate how the safety agent operates when thresholds are exceeded:

- **Temperature (vegetative stage):** if the temperature rises above 28 °C, the agent detects the deviation, logs the event, and generates an alarm. A corrective action is triggered through the control system, such as activating the ventilation system or reducing light intensity. This ensures rapid stabilization of the microclimate.
- **Relative humidity (flowering stage):** when humidity drops below 45%, the agent generates an alarm and activates the humidification system to restore appropriate levels. Maintaining adequate humidity reduces the risk of water stress and prevents negative impacts on flower development.
- **CO_2 concentration (flowering stage):** if CO_2 levels fall below 400 ppm, the agent initiates CO_2 injection. This corrective action optimizes photosynthesis efficiency and maintains biomass accumulation during critical growth phases.
- **pH in hydroponics:** if the nutrient solution pH drifts outside the 5.8–6.3 range, the agent logs the event and sends a corrective instruction, such as dosing with acid or base, to restore the optimal nutrient-uptake range.
- **Electrical conductivity (EC):** if EC exceeds 2.5 mS/cm during the flowering stage, the agent triggers an alarm and recommends diluting the nutrient solution with water. This prevents osmotic stress and maintains balanced nutrient absorption.
- **Irrigation flow:** if the emitter flow rate drops below 0.5 L/h, the agent generates an alarm and activates maintenance protocols (e.g., flushing irrigation lines). This action prevents under-irrigation and ensures uniform water distribution across the crop.

These examples demonstrate how the agent integrates continuous monitoring, alarm generation, and corrective actions into a closed-loop control strategy. By responding promptly to deviations, the system reduces risks associated with environmental stress, enhances crop reliability, and supports compliance with good agricultural and cultivation practices.

3.2.1.1. Quantitative reliability evaluation of the S2-E-Agent. In addition to the policy-based validation described above, a quantitative reliability study was carried out to assess the consistency and correctness of the S2-E-Agent decisions. The evaluation was implemented in a Python/Google Colab notebook that reads a CSV file with synthetic but agronomically plausible test cases and sends HTTPS `POST` requests to the production webhook of the S2-E-Agent deployed in n8n. Each test case corresponds to a specific combination of crop stage (`Vegetative`, `Flowering`, `Harvest`) and environmental/nutrient conditions, with an associated ground-truth label for both `status` (`OK`, `WARNING`, `ALARM`) and `severity` (`low`, `medium`, `high`). The test set covers normal operating conditions (`OK`), slightly out-of-range situations (`WARNING`), and clearly unsafe situations (`ALARM`).

The CSV file used in the experiments contains $N = 36$ test cases, evenly distributed across the three cultivation stages (12 per stage). For each case, the Colab script constructs a JSON payload with the five input variables (`stage`, `temperature`, `rh`, `ec`, `ph`) and sends it to the S2-E-Agent via an HTTPS `POST` request. The JSON response is parsed and the predicted `status` and `severity` are logged together with the original test case and the textual explanation returned by the agent. This process is repeated $R = 3$ times per test case with identical inputs and a short delay between calls, in order to measure both accuracy (first attempt per case) and stability (consistency across repetitions).

Table 10 summarizes the main quantitative results. Considering only the first attempt per test case, the S2-E-Agent achieves an $Accuracy_{status} = 0.81$ for the discrete risk level and $Accuracy_{severity} = 0.81$ for the severity level. Stability is computed as the proportion of test cases for which all $R = 3$ repetitions produce exactly the same prediction. The observed values are $Stability_{status} = 0.97$ and $Stability_{severity} = 0.97$, indicating a highly consistent behaviour when the input conditions remain unchanged.

The confusion matrix for the risk status (Table 11) shows that the agent correctly classifies 8 out of 9 `OK` cases, 12 out of 18 `WARNING` cases, and all 9 `ALARM` cases. Most misclassifications occur in borderline situations between `OK` and `WARNING` or between `WARNING` and `ALARM`, which is

Table 10
Global reliability metrics for the S2-E emergency agent. Accuracy is computed using the first attempt per case ($R = 1$). Stability is computed over $R = 3$ repetitions per case.

Metric	Status	Severity
Number of test cases N	36	36
Repetitions per case R	3	3
Accuracy	0.81	0.81
Stability (all R equal)	0.97	0.97

Table 11
Confusion matrix for the S2-E-Agent risk `status`. Rows correspond to expected labels and columns to predicted labels (first attempt per case).

Expected \ Predicted	OK	WARNING	ALARM
OK	8	1	0
WARNING	4	12	2
ALARM	0	0	9

coherent with the continuous nature of the underlying agronomic variables and with the conservative design of the decision thresholds. A per-stage analysis (Table 12) shows accuracies of 0.83 for `Vegetative`, 0.75 for `Flowering`, and 0.83 for `Harvest`, i.e., relatively uniform performance with a slight degradation in the flowering stage due to a higher proportion of borderline scenarios. A similar confusion pattern is observed for the severity levels (Table 13), confirming that the agent is able to discriminate between low, medium, and high risk in a manner consistent with the expert-defined ranges.

Overall, these results indicate that the S2-E-Agent provides a reliable first layer of risk assessment for controlled-environment cannabis cultivation. The combination of reasonably high accuracy (above 0.8 for both status and severity), perfect detection of `ALARM` situations, and very high stability (≈ 0.97) is particularly relevant for safety-critical scenarios, where false negatives in high-risk conditions are more problematic than occasional conservative warnings. The observed errors are concentrated in borderline cases between `OK`, `WARNING`, and `ALARM`, which could be further reduced by refining the prompt, introducing cost-sensitive thresholds, or incorporating additional contextual variables (e.g., duration of exposure or recent trends in the variables). It is important to note that the current evaluation is based on a synthetic test set derived from expert rules; future work will extend this analysis with real-world data from greenhouse deployments and will compare different LLM backends and calibration strategies. Despite these limitations, the evidence suggests that the proposed agent is suitable as a decision-support component and can be safely integrated into the ASA pipeline, especially when combined with human supervision and complementary monitoring agents.

3.2.1.2. Error analysis of WARNING→OK misclassifications. Table 11 shows four cases where expected `WARNING` conditions were predicted as `OK`. Manual inspection of these test cases indicates that the errors were predominantly *boundary cases* close to warning thresholds, where minor sensor noise, rounding, or short transients can shift the classification. In addition, some cases involved partial contextual cues (e.g., missing a secondary variable that would support a warning) in the minimal JSON payload. To reduce these errors, we propose: (i) adding a calibrated safety margin around thresholds (gray-zone mapping to `WARNING`); (ii) requiring persistence across multiple consecutive readings for warnings (as in Section 2.3.1); and (iii) augmenting the prompt with explicit stage context and validated sensor aggregates.

3.3. Benchmark

To assess both performance and functionality, we conducted a benchmark using 10 structured JSON payloads emulating sensor readings in different cultivation stages (vegetative, flowering, and harvest). Each request was sent via HTTP `POST` to the n8n-based S2-IoT agent, and standard latency metrics were collected using the `pycurl` library in Python. These metrics include DNS lookup time (t_{dns}), connection time (t_{connect}), time to first byte (TTFB, t_{ttfb}), and total response time (t_{total}).

Table 12
Per-stage accuracy of the S2-E-Agent on the risk `status` (first attempt per case).

Stage	Number of cases	Accuracy (status)
Vegetative	12	0.83
Flowering	12	0.75
Harvest	12	0.83

Table 13
Confusion matrix for the S2-E-Agent `severity` level. Rows correspond to expected labels and columns to predicted labels (first attempt per case).

Expected \ Predicted	low	medium	high
low	8	1	0
medium	4	12	2
high	0	0	9

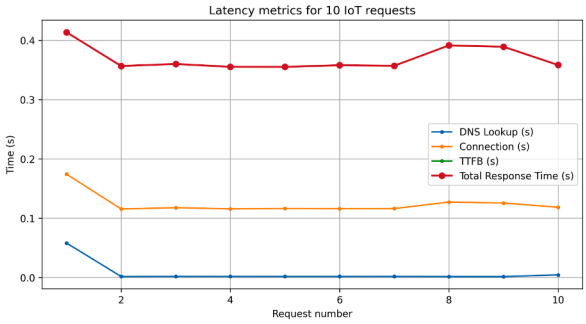


Fig. 14. Latency metrics for 10 S2 emergency and IoT requests, including DNS lookup (Domain Name System lookup), connection time, TTFB (Time To First Byte), and total response time.

Fig. 14 shows the distribution of response times across the 10 requests. The total response time remained consistently below one second, with an average latency of **0.369 s** and a 90th-percentile latency of **0.393 s**. These results confirm that the S2-IoT agent achieves low and stable response times, making it suitable for real-time monitoring and decision support in controlled environments.

In addition to latency, we analyzed the functional correctness of the responses. Table 14 presents the input payloads (questions) together with the corresponding outputs (agent responses). In the vegetative stage, the agent flagged relative humidity values above 80% as *out of range*, reporting the numerical difference from the expected reference. During the flowering stage, both RH and temperature deviations were identified and clearly reported in the JSON response. At the harvest stage, the agent highlighted low humidity levels as problematic. This confirms that the S2-IoT agent not only ensures timely responses but also integrates contextual agronomic knowledge to provide actionable insights. For convenience, Table 15 summarizes all acronyms used in the paper.

4. Discussion

This section discusses the empirical results and their implications for integrating LLM-based agents as operational services in smart greenhouse IoT workflows.

Table 14
Summary of S2-E-Agent test requests and decisions.

Req #	Context	RH / Temp	Agent decision
1	Vegetative	90% / 20°C	Unsafe: strong high-humidity alarm.
2	Vegetative	75% / 21°C	Warning: humidity above range, increase ventilation.
3	Vegetative	80% / 22°C	Unsafe: high humidity, corrective action required.
4	Vegetative	85% / 23°C	Unsafe: very high humidity, urgent alarm.
5	Flowering	60% / 24°C	Safe but close to upper RH limit.
6	Flowering	55% / 25°C	Safe: RH and temperature within range.
7	Flowering	50% / 26°C	Warning: high temperature, humidity OK.
8	Harvest	45% / 23°C	Mild warning: RH near lower bound.
9	Harvest	40% / 22°C	Alarm: low humidity, recommend humidification.
10	Harvest	35% / 21°C	Unsafe: very dry conditions, strong alarm.

Table 15
Acronyms used in the paper.

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASA	Agent-Based Service Architecture
BLE	Bluetooth Low Energy
BLEURT	Bilingual Evaluation Understudy with Representations from Transformers (semantic evaluation metric)
Chat-Agent (S1)	User-facing conversational agent (ASA service)
CO_2	Carbon Dioxide
CSV	Comma-Separated Values
CoAP	Constrained Application Protocol
DB	Database (e.g., InfluxDB time-series store)
DLI	Daily Light Integral
DNS	Domain Name System
EC	Electrical Conductivity
EM	Exact Match (task-level accuracy metric)
ESP	Espressif microcontroller family (ESPxx)
ET	Evapotranspiration
GMP	Good Manufacturing Practice
GPT-4	Generative Pre-trained Transformer 4 (OpenAI large language model)
GPT-5	Generative Pre-trained Transformer 5 (OpenAI large language model)
GPU	Graphics Processing Unit
HA	Home Assistant
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IPC	Industrial PC
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
LoRaWAN	Long Range Wide Area Network
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MQTT	Message Queuing Telemetry Transport
NFT	Nutrient Film Technique (hydroponics)
OPC UA	Open Platform Communications - Unified Architecture
PAR	Photosynthetically Active Radiation
PID	Proportional-Integral-Derivative (control)
PPFD	Photosynthetic Photon Flux Density
PV	Photovoltaic
PoC	Proof of Concept
QA	Question Answering
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RH	Relative Humidity
RL	Reinforcement Learning
RMSE	Root Mean Squared Error
ROUGE-L	Longest common subsequence variant of the ROUGE metric
S2-E-Agent	Emergency actuation agent (ASA service)
S2-IoT-Agent	Telemetry-driven monitoring/actuation agent (ASA service)
SOP	Standard Operating Procedure
TCP	Transmission Control Protocol (as in Modbus/TCP)
TTFB	Time To First Byte
UI	User Interface
VPD	Vapor Pressure Deficit
VPS	Virtual Private Server
Wi-Fi	Wireless Fidelity (IEEE 802.11)

4.1. S1 Chat-Agent reliability under RAG grounding

We begin the Discussion by interpreting the quantitative Natural Language Processing evaluation of the S1 Chat-Agent reported in Table 7. Using the 40-question benchmark in Table 6, **GPT-5.2 shows the best overall performance** compared with GPT-4o-Mini. At the aggregate level, GPT-5.2 increases ROUGE-L from 0.198 to 0.272 (+37% relative) and BLEU-4 from 3.410 to 3.821 (+12% relative), while BERTScore remains essentially unchanged (0.872 vs. 0.874; −0.002 absolute, i.e., within typical metric variability). Most importantly, the manual outcome labels reveal a substantial reliability gain: GPT-5.2 answers 38 out of 40 questions correctly (95.0% accuracy), with only one erroneous and one not-found case, whereas GPT-4o-Mini answers 32 out of 40 correctly (80.0% accuracy), with three erroneous and five not-found cases. This indicates that GPT-5.2 not only improves lexical overlap with the reference answers, but also markedly reduces retrieval-related failures (**NF**) in the RAG-based setting.

The S1 results suggest that a RAG pipeline grounded on the greenhouse knowledge base can successfully handle factual agronomic questions and numeric queries in routine operational use. In practice, this supports the use of the Chat-Agent for consultation and reporting tasks, while still motivating initial validation and periodic spot checks by domain experts—particularly when the knowledge base is updated or when operating conditions change.

4.2. Why lexical metrics remain modest in explanatory agronomic QA

Interpreting ROUGE-L, BLEU-4, and BERTScore. ROUGE-L measures longest-common-subsequence overlap and BLEU-4 measures modified n-gram precision up to 4-grams; both are lexical metrics and therefore are expected to be modest for open-ended, explanatory answers where multiple correct phrasings exist. Consequently, ROUGE-L values around 0.20–0.27 and BLEU-4 values around 3–4 should be interpreted primarily *comparatively* across models under identical conditions, rather than against absolute thresholds. In contrast, BERTScore captures semantic similarity using contextual embeddings and is more robust to paraphrasing; the ≈ 0.87 values observed here indicate strong semantic agreement with the reference answers even when exact wording differs. Taken together, the metrics and the outcome labels consistently support GPT-5.2 as the strongest model in our evaluation.

4.3. S2-IoT: Validating end-to-end telemetry computation and daily reporting

From a technological standpoint, the reported results indicate that Large Language Model (LLM)-based agents can be reliably integrated into greenhouse management workflows when they are embedded in a carefully designed service-oriented architecture. The ASA model introduced in this work separates sensing, reasoning, and actuation concerns, and encapsulates them into modular services (S1, S2-IoT-Agent, S2-E-Agent) orchestrated through a low-code workflow engine. This separation is crucial to enforce operational constraints, to reuse components across use cases, and to evolve individual services without disrupting the whole system.

The analysis of the S2-IoT-Agent confirms that LLM-based agents can also be used as a programmable analysis layer on top of IoT datasets. By standardizing prompts and expected outputs, the agent is able to compute descriptive statistics, detect anomalies, segment periods of interest, and derive consumption indicators from time-series data exported by the greenhouse platform. The near-zero numerical errors observed when compared with ground-truth calculations (e.g., DLI estimation in Table 8) suggest that, under controlled prompting and constrained tasks, LLMs can act as a flexible “soft computation” layer, reducing the need to hard-code every analytic query. This is particularly attractive in contexts where data schemas or agronomic requirements evolve over time.

4.4. S2-E: Constrained emergency reasoning and safety policies

Regarding the S2-E-Agent, the combination of stage-dependent safety policies and event-driven orchestration enables context-aware responses to potentially unsafe situations. Experiments with synthetic and real emergency scenarios show that the agent can produce consistent alarms and recommendations with sub-second total response times, even when deployed on a low-cost VPS. Compared with a basic rule-based controller, the ASA layer offers richer reasoning over multiple variables and crop stages, while still enforcing explicit thresholds and safety margins. However, this additional intelligence comes at the cost of higher implementation complexity and a strict need for rigorous testing and monitoring before enabling fully automatic actuation.

From a practical point of view, the quantitative results obtained for the S2-E-Agent are encouraging. An accuracy slightly above 0.8, combined with perfect detection of `ALARM` situations and very high stability across repetitions, is consistent with the role of a first-line decision-support component rather than a fully autonomous controller. In other words, the agent behaves conservatively in borderline situations (sometimes promoting cases from `OK` to `WARNING` or from `WARNING` to `ALARM`), which is acceptable—and arguably desirable—in safety-critical greenhouse scenarios, where missing a truly dangerous condition is more costly than issuing an occasional false alarm.

At the same time, the fact that most misclassifications are concentrated around the decision boundaries suggests clear directions for improvement (e.g., refining expert thresholds, incorporating temporal trends, or calibrating the LLM outputs), without questioning the overall suitability of the agent for integration into the ASA architecture under human supervision.

4.5. Resilience: Offline mode, missed alarms, and auditability

A key operational requirement is maintaining safe behavior during connectivity loss. In our implementation, cloud-level workflows (e.g., scheduled agent calls, reporting, and cross-service coordination) run in `n8n` on a VPS, while time-critical monitoring and bounded local logic run in `Node-RED/Home Assistant` at the edge. When cloud orchestration is unavailable, the system falls back to an edge-first offline mode: local sensing and essential control loops continue running, telemetry and events are buffered with timestamps for later synchronization, and operators are notified of the offline state. Upon reconnection, buffered events are uploaded as historical records to preserve auditability and enable catch-up notifications; importantly, the system does not retroactively actuate based on historical alarms, and instead re-evaluates the current state before issuing new actions or recommendations. This design supports continuity of operation while keeping actions traceable and aligned with explicit policies.

4.6. Generalizability and deployment scope

Scope note: although the architecture is designed to be modular and transferable, the experimental validation reported in this paper was performed in a single 50 m^2 hydroponic industrial hemp greenhouse. Accordingly, statements about generality should be interpreted as *design intent* supported by modularity, rather than as empirically demonstrated performance across multiple crops and production modalities.

The architecture is intended to be transferable because it separates (i) the physical sensing/actuation layer, (ii) the operational services and orchestration layer, and (iii) the knowledge base and agent constraints. In this design, adaptation to other crops or cultivation modalities primarily affects the sensor/actuator configuration, the SOP/knowledge-base content used for RAG grounding, and a bounded subset of rules and thresholds used by the operational services; the service interfaces, audit/logging patterns, and workflow structure remain stable.

4.6.1. Adaptability to a distinct cultivation scenario

To make the adaptability claim more concrete, we outline how the same ASA can be adapted to a soil-based tomato greenhouse. In this scenario, the *Physical Layer* would replace hydroponic probes (pH/EC/water temperature) with soil moisture, soil temperature, and soil nutrient sensors (or ion-selective probes), and would adapt actuator mappings for drip irrigation and fertigation. The *Knowledge Base* used by the S1 Chat-Agent would ingest tomato-specific SOPs and cultivation manuals (e.g., pruning, trellising, pest scouting, irrigation set-points), while the *S2-IoT-Agent* rule set would be updated to reflect soil water-holding dynamics (e.g., sensor thresholds, irrigation pulse logic, leaching fractions) and tomato growth-stage constraints. The orchestration and service layers (data pipelines, agent interfaces, logging, and user interaction patterns) remain unchanged, illustrating that adaptation mainly affects crop-/process-specific sensing, SOP content, and a bounded subset of agent rules.

4.7. Limitations and next steps

Several limitations must be acknowledged. First, the evaluation relies on a single experimental greenhouse and on a limited number of data-driven and emergency scenarios, which constrains the generalizability of the findings. Second, only OpenAI-based LLMs were tested, so the results cannot be extrapolated to other providers without additional benchmarking. Third, the tuning of prompts, thresholds, and workflows was carried out by the authors and has not yet been validated with independent growers or technicians. Despite these limitations, the reported results provide a realistic picture of what can be achieved with current LLM technology in operational IoT settings, and they highlight the importance of combining AI agents with explicit policies, logging, and human oversight instead of aiming for fully autonomous control.

Finally, the experience gained in this case study suggests that low-code orchestration and open-source IoT platforms are key enablers for bringing AI agents closer to small and medium growers. By relying on mainstream tools, the proposed ASA can be replicated and adapted to other crops or facilities with modest engineering effort, turning the architecture into a reusable starting point for practical blueprint rather than a purely conceptual proposal.

5. Conclusion

This paper has presented the design, deployment, and validation of an Agent-Based Service Architecture (ASA) that integrates IoT sensing, low-code orchestration, and LLM-based agents to support precision agriculture in a hydroponic greenhouse. The proposed architecture was instantiated in a 50 m^2 experimental installation instrumented with environmental and fertigation sensors and connected to a commercial-grade control platform. On top of this infrastructure, three main services were implemented: a conversational Chat-Agent (S1), a telemetry-driven analysis agent (S2-IoT-Agent), and an emergency agent (S2-E-Agent).

Quantitative and qualitative results show that the ASA can reliably support typical decision-making tasks in controlled cultivation environments. The S1 Chat-Agent, backed by a RAG pipeline, is able to answer factual and numeric agronomic questions with high accuracy while providing human-readable explanations. The S2-IoT-Agent reproduces ground-truth computations on exported IoT datasets for water balance and anomaly detection, effectively acting as a programmable analytics layer. The S2-E-Agent enforces stage-dependent safety policies on real-time telemetry and issues consistent alarms and recommendations with low response times, demonstrating that LLM-based agents can participate in time-sensitive monitoring workflows when appropriately constrained.

From a technological perspective, the case study confirms that advanced decision-support services for growers can be built by combining mainstream open-source IoT platforms with state-of-the-art LLM APIs and low-code orchestration tools, without requiring complex custom software stacks. The ASA model thus emerges as a flexible, low-cost, and open-source-friendly blueprint that can be adapted to other crops, greenhouse typologies, or even non-agricultural industrial environments. By encapsulating AI agents as services with well-defined interfaces, and by grounding them on explicit policies and standardized prompts, the architecture facilitates maintenance, incremental upgrades of the underlying models, and long-term accumulation of agronomic knowledge.

Overall, the proposed ASA demonstrates that LLM-based agents can move beyond isolated prototypes and become first-class components of operational IoT systems for precision agriculture. While further validation at scale and under more diverse conditions is still needed, the reported results provide a concrete pathway for researchers, technologists, and integrators interested in leveraging AI agents as practical digital companions for sustainable, data-driven crop management.

5.1. Future work

Building on the findings of this work, several directions for future research are identified. First, further optimization of the AI layer is required, particularly in vectorization and knowledge-retrieval processes. Enhancing these components should improve the accuracy, efficiency, and adaptability of the agents across multiple functionalities. Second, systematic benchmarking of large language models (LLMs) beyond the OpenAI ecosystem should be conducted, as this may reveal alternative models with improved reasoning capabilities, lower costs, or domain-specific advantages. Third, the integration of broader IoT data streams—including advanced imaging, spectral sensing, and plant-physiology indicators—should be explored to enable richer datasets for AI-driven decision-making.

Another avenue for improvement lies in the design of standardized prompt strategies and the development of curated prompt libraries. These resources would help unify analytical processes, ensure consistency of reports, and facilitate long-term comparative studies across different crops, cultivation methods, and temporal windows. Additionally, refining the emergency response agent (S2-E-Agent) through the incorporation of adaptive and learning-based policies could strengthen its capacity to handle unforeseen events beyond predefined rules.

Finally, large-scale validation with multiple growers and in diverse greenhouse settings will be necessary to assess generalizability and user acceptance. Such efforts will not only improve the robustness of the ASA model but also accelerate its transfer into commercial agricultural practice, positioning it as a reliable digital assistant for sustainable, data-driven cultivation.

Ethical statement

This research did not involve human participants or animals. All experiments were conducted in a controlled greenhouse environment following standard agronomic practices and safety regulations; therefore, ethical approval was not required.

CRediT authorship contribution statement

Sofía Pardo-Pina: Validation, Supervision, Methodology, Investigation; **Jörn Germer:** Visualization, Validation, Supervision; **Ricardo Suay-Cortés:** Visualization, Validation, Investigation; **Manuel Platero-Horcajadas:** Writing – review & editing, Software, Formal analysis, Conceptualization; **Francisco-Javier Ferrández-Pastor:** Writing – original draft, Software, Methodology, Formal analysis, Conceptualization.

Data availability

The data that has been used is confidential.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors thank the growers and technical staff who supported the greenhouse deployment and day-to-day operations. We also acknowledge the PURECIRCLES consortium for facilitating access to the hydroponic setup, SolarCloth for enabling the pilot installation of the PV shading mesh, and the open-source communities behind Home Assistant and n8n for their software and documentation. We are grateful to the agronomists and engineers at the Plant Experimentation Unit of the University of Alicante campus, who provided constructive feedback during validation, and to the students who assisted with data processing and dashboard development.

References

[1] J. Hansen, H. Dong, Design of an Internet of Things (IoT)-Based Photosynthetically Active Radiation (PAR) Monitoring System, AgriEngineering 6 (1) (2024) 773–785. https://doi.org/10.3390/agriengineering6010044

[2] A. Simo, S. Dzitac, G. Badea, D. Meianu, Smart Agriculture: IoT-based Greenhouse Monitoring System, International Journal of Computers Communications and Control (2022) https://doi.org/10.15837/ijccc.2022.6.5039.

[3] R. Al-Qudah, M. Almuhajri, C.Y. Suen, Unveiling the potential of sustainable agriculture: a comprehensive survey on the advancement of AI and sensory data for smart greenhouses, Comput. Electron. Agric. 229 (2025) 109721.

[4] D. Muhammed, E. Ahvar, S. Ahvar, M. Trocan, M.-J. Montpetit, R. Ehsani, Artificial Intelligence of Things (AIoT) for smart agriculture: a review of architectures, technologies and solutions, J. Netw. Comput. Appl. (2024) https://doi.org/10.1016/j.jnca.2024.103905.

[5] J. Lee, S. Im, J.S. Jeong, T.S. Lee, S.H. Park, C. Shin, H.J. Kim, Learning hidden relationship between environment and control variables for direct control of automated greenhouse using transformer-based model, Comput. Electron. Agric. 235 (2025) 110335.

[6] S. Mansoor, S. Iqbal, S.M. Popescu , S.L. Kim, Y.S. Chung and J.H. Baek , Integration of smart sensors and IOT in precision agriculture: trends, challenges and future prospectives, (2025), Front. Plant Sci https://doi.org/10.3389/fpls.2025.1587869

[7] International Society of Precision Agriculture, Precision Ag Definition, 2024, (Website). Accessed: 2026-01-09, https://www.ispag.org/.

[8] M.M. del Carmen Arreola, E.Z. Lau, Precision agriculture in the age of AI and nanotechnology: global trends and latin america's transition, in: M.M. Erdoğdu, E.Z. Lau, A.C. García (Eds.), Artificial Intelligence & the Future of Humanity, IJOPEC Publication Limited, (2025), pp. 187–208.

[9] Sustainability Directory, AI Driven Urban Agriculture Resilience, Scenario (2025). https://www.scenariomag.com/ai-driven-urban-agriculture-resilience.

[10] A. Alkamely, W.M ELMedany, AI-IoT-enabled controlled environment agriculture, Institution of Engineering and Technology (2021) 296–301. https://doi.org/10.1049/icp.2021.0858

[11] S.M. Shekarian, M. Aminian, A.M. Fallah, V. Akbary Moghaddam, AI-Powered sensor fault detection for cost-effective smart greenhouses, Comput. Electron. Agric. 224 (2024) 109198. https://doi.org/10.1016/j.compag.2024.109198

[12] E. Bicamumakuba, M.N. Reza, H. Jin Samsuzzaman, K.-H. Lee, S.-O. Chung, Multi-Sensor monitoring, intelligent control, and data processing for smart greenhouse environment management, Sensors 25 (19) (2025) 6134. https://doi.org/10.3390/s25196134

[13] B. Morcego, W. Yin, S. Boersma, E. van Henten, V. Puig, C. Sun, Reinforcement learning versus model predictive control on greenhouse climate control, Comput. Electron. Agric. 215 (2023) 108372. https://doi.org/10.1016/j.compag.2023.108372

[14] S. Sawant, R. Nair, S. Hariharan, Empowering farmers with artificial intelligence: a retrieval-augmented generation based large language model advisory framework, J. Agric. Eng. (2026). Early Access / Preprint. https://doi.org/10.4081/jae.2026.1908

[15] M.T. Hasan, A.A. Khan, M. Saari, V. Bankhele, P. Abrahamsson, Towards AI Evaluation in Domain-Specific RAG Systems: The AgriHubi Case Study, (2026), https://doi.org/10.48550/arXiv.2602.02208

[16] R. Al-Najadi, Y. Al-Mulla, K. Goher, Advances in intelligent and autonomous greenhouse systems: a comprehensive review of internet of things, artificial intelligence, and robotics integration, Smart Agric. Technol. 13 (2026) 101670. https://doi.org/10.1016/j.atech.2025.101670

[17] G.P.C. Tancredi, L. Preite, G. Vignali, Digital twin enhanced with machine learning algorithms for irrigation management using sensor data, Procedia Comput. Sci. 253 (2025) 2419–2427. https://doi.org/10.1016/j.procs.2025.01.302

[18] C. Catalano, L. Paiano, F. Calabrese, M. Cataldo, L. Mancarella, F. Tomassi, Anomaly detection in smart agriculture systems, Computers in Industry 143 (2022) 103750. https://doi.org/10.1016/j.compind.2022.103750

[19] Y. Filchenkova, IoT In greenhouse automation: controlling climate with smart sensors, Fordewind.io (2025). https://fordewind.io/blog/iot-in-greenhouse-automation.

[20] M.H. Lee, M.H. Yao, P.Y. Kow, F.J. Chang, An Artificial Intelligence-Powered Environmental Control System for Resilient and Efficient Greenhouse Farming, Agriculture 13 (11) (2023) 2137. https://doi.org/10.3390/su162410958

[21] A.C.H. Austria, J.S. Fabros, K.R.G. Sumilang, J. Bernardino, A.C. Doctor, Development of IoT smart greenhouse system for hydroponic gardens, Int. J. Comput. Sci. Res. 7 (2023) 2111–2136.

[22] A. Renuka, AI-Driven Predictive Analytics in Precision Agriculture, in: Scientific Journal of Artificial Intelligence and Blockchain Technologies, (2025), https://doi.org/10.63345/sjaibt.v2.i3.104

[23] K. Kartheek, Low-Code BPM meets IoT: A Framework for Real-Time Industrial Automation, International Journal of Scientific Research in Engineering and Management 8 (1) (2024) 1–13. https://doi.org/10.55041/IJSREM28062

[24] E. Cabrera, Chatbot for Promoting Best Crop Management Practices to Rice Farmers in Odisha, India, in: Asian Journal of Agricultural Extension Economics & Sociology, (2024). https://doi.org/10.9734/ajaees/2024/v42i102581

[25] S.D. Putra, H. Heriansyah, E.F. Cahyadi, et al., Development of Smart Hydroponics System using AI-based Sensing, Jurnal Infotel 18 (3) (2024) 1–12. https://doi.org/10.20895/infotel.v16i3.1190

[26] R.K. Kasera, S. Gour, T. Acharjee, A comprehensive survey on IoT and AI based applications in different pre-harvest, during-harvest and post-harvest activities of smart agriculture, Comput. Electron. Agric. 216 (2024) 108522.

[27] N. Ahmed, N. Shakoor, Advancing agriculture through IoT, big data, and AI: a review of smart technologies enabling sustainability, Smart Agric. Technol. (2025) 100848.

[28] J. Yu, C. Sun, J. Zhao, L. Ma, W. Zheng, Q. Xie, X. Wei, Prediction and control of greenhouse temperature: methods, applications, and future directions, Comput. Electron. Agric. 237 (2025) 110603.

[29] J. Morales-Garcia, F. Terroso-Saenz, J.M. Cecilia, A multi-model deep learning approach to address prediction imbalances in smart greenhouses, Comput. Electron. Agric. 216 (2024) 108537.

[30] Z. Wang, Z. Liu, M. Yuan, W. Yin, C. Zhang, Z. Zhang, X. Hu, A machine learning-based irrigation prediction model for cherry tomatoes in greenhouses: leveraging optimal growth data for precision irrigation, Comput. Electron. Agric. 237 (2025) 110558.

[31] PURECIRCLES Consortium, PURECIRCLES Project (European Project), 2025, (https://purecircles.uni-hohenheim.de/en). Accessed: Aug. 20, 2025.

[32] Solar Cloth, Solar Cloth - Photovoltaic Shading Mesh (Company Website), 2025, (https://www.solar-cloth.com). Accessed: Aug. 20, 2025.

[33] R.W. Thimijan, R.D. Heins, Photometric, radiometric, and quantum light units of measure: a review of procedures for interconversion, HortScience 18 (6) (1983) 818–822.